



TECHNICAL LEARNING GUIDE

FindMyProfessor Architecture & Learning Guide

A developer manual for the current FindMyProfessor codebase
— traceable to source files, honest about gaps, and organized
as a book for onboarding, review, and interview preparation.

Generated: September 12, 2026

Source: docs/ARCHITECTURE.md & docs/architecture/*.md (14 documents)

Scope: System overview · Data model · Auth · CV pipeline · Matching engine · Email generation · Gmail OAuth & Send · Frontend/Backend · Deployment · Security · Failure scenarios · API map · Interview Q&A · Cheat sheets

Table of Contents

Every chapter below links directly to its page; page numbers reflect the final print layout.

PART I — SYSTEM OVERVIEW

1. Introduction & Purpose	3
2. System Architecture (C4 Diagrams)	4
3. Repository Map	11
4. End-to-End User Flow	12

PART II — CORE SUBSYSTEMS

5. Data Model & Database	15
6. Authentication vs. Authorization	19
7. CV Pipeline	23
8. Matching Engine	28
9. Email Generation	32
10. Gmail OAuth	36
11. Gmail Send	39
12. Frontend Architecture	42
13. Backend Architecture	44
14. Deployment Architecture	46
15. Security Architecture	48
16. Failure Scenarios	54
17. API Map	56

PART III — REFERENCE

18. Interview Q&A	58
19. Learning Roadmap	60
20. Architecture Glossary	61
21. Final Cheat Sheets	62

1. Introduction & Purpose

Source: docs/ARCHITECTURE.md, Section 1

Product purpose: FindMyProfessor helps a student find professors whose research genuinely overlaps with their own background (from their CV), and then helps them write and send a credible, personalized outreach email — without fabricating claims about either the student or the professor.

Main user journey: sign in → upload a CV → review/confirm the extracted profile → browse ranked professor matches (and funded opportunities) → pick one → generate an email draft → edit it → connect Gmail → send, with an explicit confirmation step.

Major Subsystems

- **Frontend** — React 18 SPA, Vite build, Supabase JS client for auth
- **Backend** — FastAPI, no ORM, talks to Postgres exclusively through the Supabase query builder
- **Database** — Supabase-hosted Postgres; base catalog schema lives outside version control, four incremental migrations are tracked
- **Authentication** — Supabase Auth (Google OAuth) + a legacy client-supplied profile-id header, mid-hardening
- **CV processing** — format-specific text extraction, then heuristic (default) or optional LLM structured-field extraction
- **Matching engine** — deterministic, rule-based scoring; zero ML/embeddings
- **Outreach/email generation** — deterministic template provider (default) or optional LLM provider, both fed only verified/matched evidence
- **Gmail OAuth** — gmail.send-only scope, encrypted token storage
- **Gmail API** — actual send call, MIME construction, CV attachment
- **Deployment infrastructure** — inferred to be Render (backend) + a custom domain (frontend host unconfirmed); no deployment manifest is checked into this repo
- **External services** — Supabase (DB + Auth), Google OAuth/Gmail API, optionally an OpenAI-compatible LLM endpoint

The 30-Second Explanation

ELEVATOR PITCH

"FindMyProfessor is a React + FastAPI app that parses a student's CV, deterministically scores every professor in our catalog against the student's extracted research interests, and helps the student draft and send a real email through their own Gmail account — with the email generation restricted to only the facts we've actually verified, so it can't invent claims about either party. There's no ML model in the matching engine; it's rule-based and reproducible on purpose."

The 2-Minute Explanation

The backend is a FastAPI service with no ORM — every DB call goes through Supabase's query builder directly, using a service-role key, so authorization is enforced entirely in application code by filtering on `profile_id`, not by Postgres Row-Level Security. A student's CV goes through a format-specific text extractor (pypdf/python-docx/etc.) and then a structured-field parser — by default a regex/heuristic parser, with an optional LLM path behind an environment variable that's off unless you set an API key.

Once the student confirms their extracted profile, a deterministic scoring function compares their research interests, inferred signals, and project/publication evidence against every professor's tagged research areas, weighted **55/20/20/5** and renormalized across whatever signal types the student actually has — so a thin CV isn't unfairly penalized. The exact same scoring function is reused when generating an outreach email, so the email's claims about "shared research interest" are the same claims the matching UI already showed the student, not a second, independently-generated set of claims.

Email generation itself is provider-abstracted: a template-based deterministic provider by default, or an LLM provider if configured — and if the LLM call fails, it silently falls back to the deterministic provider rather than erroring. The anti-hallucination strategy is entirely about restricting what data reaches the prompt (only verified, matched fields — never the raw CV, never the professor's free-text bio) plus system-prompt instructions; there's no post-generation fact-check.

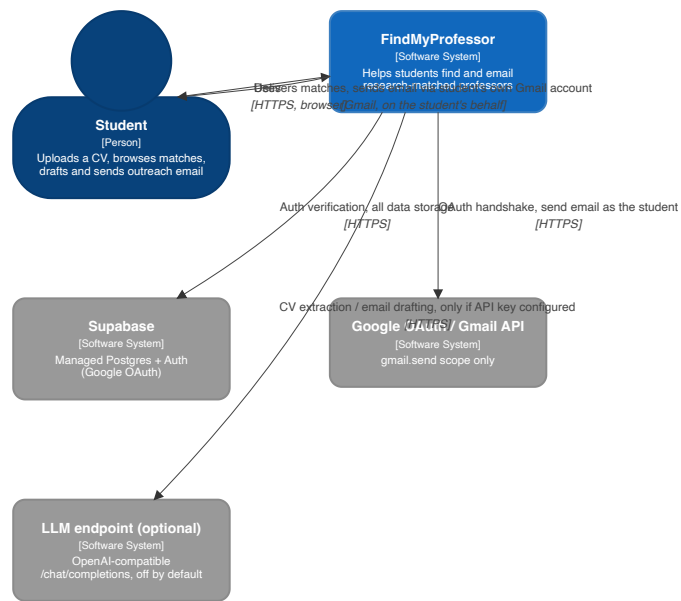
Authentication is Supabase Auth via Google OAuth, but the app has an older client-supplied `X-Profile-Id` header design underneath it, and the code is mid-way through hardening that: once a real session is presented, that header can no longer reach a different account's data, but a request with no session at all still trusts the header outright — that's a documented, known gap, not a hidden one.

Sending is entirely manual and single-shot: an explicit `confirmed:true` flag is required, and an atomic compare-and-swap on the draft's status prevents double-sends. There's no scheduler, no automatic follow-up, and the Gmail OAuth scope is `gmail.send` only — the app can never read the student's actual inbox.

2. System Architecture (C4 Diagrams)

Source: docs/ARCHITECTURE.md, Section 2

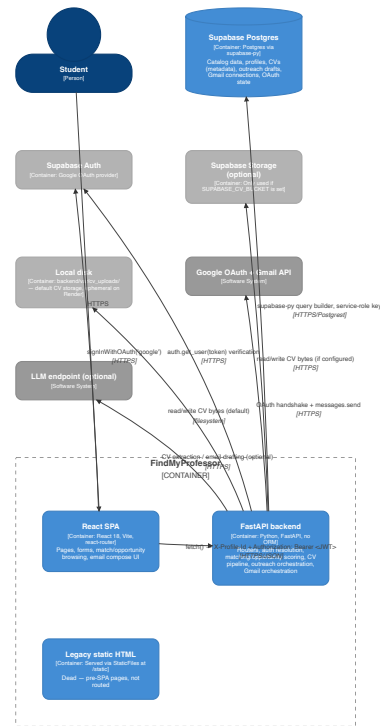
System Context



System Context diagram — docs/ARCHITECTURE.md §2

In plain English: a student only ever talks to the FindMyProfessor system. Behind the scenes, that system depends on three external services: Supabase for identity and all persistent data, Google for both login (via Supabase's Google provider) and the actual sending of email, and — only if explicitly configured — an LLM endpoint for two optional smart features. The system never emails the professor "as itself" — it always sends through the student's own authorized Gmail account.

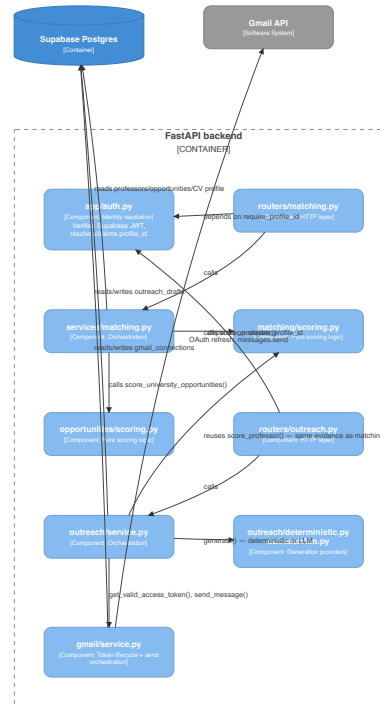
Container Diagram



Container diagram — docs/ARCHITECTURE.md §2 (required 'system architecture diagram')

In plain English: there are really only two live containers — the React SPA and the FastAPI backend. The backend is a single process talking to five external dependencies (Supabase DB, Supabase Auth, Google, an optional LLM, and either local disk or Supabase Storage for files). The "legacy static HTML" container is included for completeness but is inert — see Chapter 14 (Deployment).

Component Diagram — Backend, Matching + Outreach Area



Component diagram — docs/ARCHITECTURE.md §2

In plain English: the matching engine and the outreach/email system are not two independent implementations of "does this student fit this professor" — the outreach service directly reuses the matching engine's `score_professor()` function to build its evidence. This is a deliberate consistency guarantee: whatever the matching UI told the student about their overlap with a professor is exactly what the generated email is allowed to claim.

3. Repository Map

Source: docs/ARCHITECTURE.md, Section 3

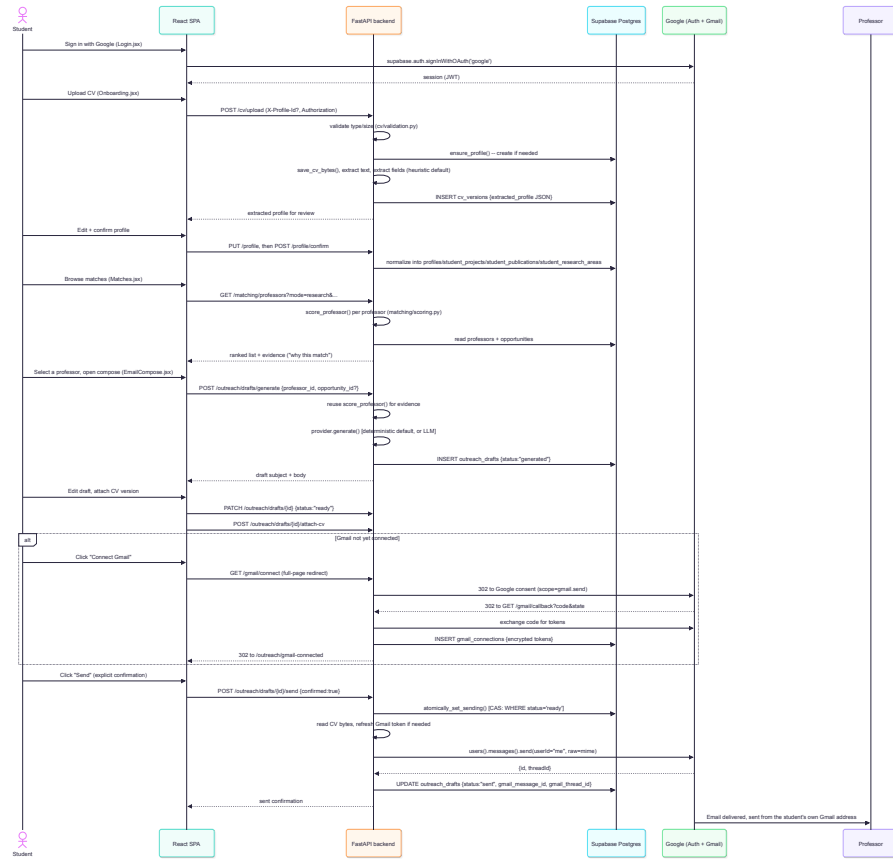
```
FindMyProfessor/
├── backend/
│   ├── app/
│   │   ├── main.py          # FastAPI app, CORS, router registration
│   │   ├── auth.py          # Identity resolution – the single most important file for security
│   │   ├── supabase_client.py # One shared Supabase client, service-role key
│   │   ├── routers/         # HTTP surface only, one file per resource
│   │   ├── schemas/         # Pydantic request/response contracts
│   │   ├── services/        # Business logic for catalog + profile domain
│   │   ├── matching/        # Pure scoring logic (no I/O) – scoring.py, normalize.py, evidence.py
│   │   ├── opportunities/    # Pure opportunity scoring logic
│   │   ├── cv/              # parsers/ (per-format text extraction), extraction/ (heuristic|LLM), storage.py, validation.py
│   │   ├── outreach/        # service.py orchestration, deterministic.py/llm.py providers, db_store.py
│   │   ├── gmail/           # oauth.py, client.py, crypto.py, service.py
│   │   ├── ingestion/       # /data JSON → Supabase, manual/one-time, not part of request handling
│   │   └── static/          # Legacy pre-SPA HTML – dead, see deployment chapter
│   ├── migrations/         # Only 4 files – incremental hardening migrations, NOT the full schema
│   ├── tests/              # pytest – read these to see real expected behavior
│   └── scripts/seed_data.py # Manual entrypoint for the ingestion pipeline
├── frontend/
│   └── src/
│       ├── App.jsx         # Route table
│       ├── pages/          # One component per route
│       ├── services/       # api.js (identity headers!), gmail.js, matching.js, outreach.js, profile.js
│       ├── hooks/          # useAuth, useGmailStatus, useProfileId
│       ├── lib/supabase.js  # Supabase client (hardcoded URL/anon key)
│       └── components/     # AppShell, GmailWidget, ProfessorCard, MatchScore, etc.
└── data/                  # Source-of-truth JSON catalog: universities/departments/labs/professors/opportunities/research_areas
```

Where to Look for What

- Identity/security → backend/app/auth.py + Chapter 6 (Auth vs Authorization)
- "Why did this score come out this way" → backend/app/matching/scoring.py + Chapter 8 (Matching Engine)
- "Why didn't my CV parse" → backend/app/cv/ + Chapter 7 (CV Pipeline)
- "Why did sending fail" → backend/app/gmail/ + backend/app/outreach/service.py + Chapter 11 (Gmail Send)

4. End-to-End User Flow

Source: docs/ARCHITECTURE.md, Section 4



End-to-end user flow — docs/ARCHITECTURE.md §4

Step-by-Step Reference Table

Step	Frontend	Endpoint	Router	Service	DB	Example API
Sign in	Login.jsx	—	—	—	auth.users (Supabase-managed)	Google Sign Up
Upload CV	Onboarding.jsx	POST /cv/upload	routers/cv.py	services/cv.py::upload_and_parse	cv_versions insert	no LL code
Confirm profile	Onboarding.jsx	PUT /profile, POST /profile/confirm	routers/profile.py	services/cv.py	profiles, student_projects, student_publications, student_research_areas	no
Browse matches	Matches.jsx	GET /matching/professors	routers/matching.py	services/matching.py	reads professors, opportunities	no
Generate email	EmailCompose.jsx	POST /outreach/drafts/generate	routers/outreach.py	outreach/service.py	outreach_drafts insert	no LL
Edit / attach CV	EmailCompose.jsx	PATCH .../{id}, POST .../{id}/attach-cv	routers/outreach.py	outreach/service.py	outreach_drafts update	no
Connect Gmail	GmailWidget.jsx	GET /gmail/connect, GET /gmail/callback	routers/gmail.py	gmail/oauth.py, gmail/service.py	gmail_oauth_states, gmail_connections	Google OAuth
Send	EmailCompose.jsx	POST .../{id}/send	routers/outreach.py	outreach/service.py + gmail/service.py	outreach_drafts update, gmail_connections read	Google

5. Data Model & Database

Source: docs/architecture/data-model.md

Where the Schema Actually Lives

IMPORTANT LIMITATION

This repository does not contain a full CREATE TABLE history for the "core" tables (universities, departments, labs, professors, research_areas, professor_research_areas, opportunities, profiles, cv_versions, student_projects, student_publications, student_research_areas). The base schema was created directly against the Supabase project (dashboard/SQL editor) outside version control.

Migration file	What it does
20260908_opportunities_nullable_professor_lab.sql	Drops NOT NULL on opportunities.professor_id and opportunities.lab_id so a university-wide or lab-only opportunity can exist without a fabricated professor.
20260909_outreach_drafts_gmail.sql	Creates outreach_drafts and gmail_connections (full DDL below).
20260910_gmail_oauth_states.sql	Creates gmail_oauth_states (replaces an in-memory dict — see Chapter 10, Gmail OAuth).
20260910_profiles_linked_user_id.sql	Adds profiles.linked_user_id + a unique partial index, to support real authentication (see Chapter 6).

Everything below about the core tables' columns is reconstructed by reading every supabase.table(...) call in the backend plus the ingestion models — accurate to what the application code reads and writes, but exact column types/defaults/constraints on these specific tables are **NOT VERIFIED FROM CURRENT CODEBASE** beyond what's inferable from usage.

Entity-Relationship Diagram



Plain-English Reading of the Diagram

- The **catalog side** (universities → departments → labs → professors) is a strict tree: a professor optionally belongs to a lab, and separately to a department. professor_research_areas is a many-to-many join between professors and a shared research_areas catalog.
- Opportunities are deliberately NOT required to belong to a professor.** university_id is the only mandatory foreign key; professor_id and lab_id are optional. This lets the system represent a university-wide program without inventing a fake professor to attach it to.
- The **student side** (profiles → cv_versions/student_projects/student_publications/student_research_areas) is keyed off profiles.id, engineered to equal a real auth.users.id.
- outreach_drafts.professor_id is stored as a plain UUID **with no foreign-key constraint** to professors — the database will not stop a draft from referencing a professor id that doesn't exist; that's only checked in application code.

Table Inventory

Catalog Tables (read-heavy, written only by ingestion)

Table	Purpose	Key columns	Written by	Read by
universities	One row per participating university	id, name, country, city, website_url, description, is_active	ingestion/seeder.py (_seed_universities)	services/universities.py, professors.py (joins), main.py health check
departments	Academic departments within a university	id, university_id, name, website_url, description	seeder.py (_seed_departments)	services/departments.py, professors.py (joins)
labs	Research labs within a department	id, department_id, name, website_url, description, research_summary	seeder.py (_seed_labs)	services/labs.py, professors.py
professors	Faculty directory	id, lab_id, department_id, name, title, email, website_url, linkedin_url, biography, research_summary, recent_work_summary, recruiting_status, internship_available, ra_available, masters_available, phd_available, is_active	seeder.py (_seed_professors)	services/professors.py, services/matching.py, outreach/service.py
research_areas	Shared catalog of normalized research-area labels	id, name, description	seeder.py (_seed_research_areas)	services/cv.py (_catalog_labels), professors.py, matching.py
professor_research_areas	Many-to-many join: professor ↔ research area	professor_id, research_area_id	seeder.py	services/professors.py, services/matching.py
opportunities	Internship/RA/Master's/PhD opportunities, at professor, lab, or university level	id, professor_id (nullable), lab_id (nullable), university_id (required), title, opportunity_type, status, international_eligible, undergraduate_eligible, application_deadline, ...	seeder.py	services/matching.py, outreach/service.py

Student/Profile Tables (written by the app at runtime)

Table	Purpose	Key columns	Written by	Read by
profiles	One row per student identity; id doubles as (or links to) the Supabase auth.users.id	id, linked_user_id, full_name, email, university_name, degree, field_of_study, country, current_semester, graduation_year, research_summary, bio	services/cv.py (ensure_profile, _write_normalized_tables)	auth.py (identity resolution), services/cv.py, outreach/service.py
cv_versions	Every CV a student has uploaded, with full extracted-profile JSON in description	id, profile_id, file_name, storage_path, version_number, is_default, description (JSON)	services/cv.py	services/cv.py, outreach/service.py
student_projects	Normalized project list, written only on profile confirm	profile_id, title, description, technologies	services/cv.py	Not read back anywhere else in the backend found — NOT VERIFIED as consumed downstream
student_publications	Normalized publication list, written only on profile confirm	profile_id, title, venue, publication_year, status, description	services/cv.py	Same caveat as above

Table	Purpose	Key columns	Written by	Read by
student_research_areas	Catalog-matched research interests, with priority order	profile_id, research_area_id, priority	services/cv.py	NOT VERIFIED as read by matching — matching reads research_interests directly off the CV JSON blob, not this table

Outreach & Gmail Tables (DDL exists in repo)

Table	Purpose	Key columns	Written by	Read by
outreach_drafts	One row per generated/edited/sent outreach email	id, profile_id, professor_id (no FK), email_type, subject, body, selected_opportunity_id, cv_version_id, matched_research_areas (jsonb), evidence_used (jsonb), generation_provider, status, sent_at, gmail_message_id, gmail_thread_id, error_code, error_message	outreach/db_store.py	outreach/service.py (all outreach endpoints)
gmail_connections	One Gmail OAuth connection per profile (UNIQUE(profile_id, provider))	id, profile_id, provider, provider_account_email, access_token_encrypted, refresh_token_encrypted, token_expires_at, scopes, revoked_at	gmail/service.py	gmail/service.py, outreach/service.py
gmail_oauth_states	Short-lived CSRF state for the OAuth handshake	token (PK), profile_id, return_to, expires_at	gmail/oauth.py	gmail/oauth.py

Ingestion Pipeline: How /data/*.json Becomes Database Rows

Source of truth for the catalog data is the /data directory at the repo root — five JSON collections (universities/, departments/, labs/, professors/, opportunities/) each with one file per university (university_01.json ... university_30.json), plus a shared research_areas/catalog.json and a universities/manifest.json listing which university files are enabled.

Pydantic validation layer — backend/app/ingestion/models.py: every JSON record is parsed into a strict SeedModel subclass. Notable validation: blank strings coerced to None; opportunity_type restricted to ("internship", "ra", "masters", "phd"); professor.email, when present, must contain @ and not start/end with it; deadline/start dates parsed as ISO dates.

Entrypoint: backend/app/ingestion/seeder.py, class Seeder.run(), called in this fixed order because later entities reference earlier ones by foreign key: universities → departments → labs → research_areas → professors → professor_research_areas → opportunities.

Upsert, not blind insert. For every entity type, the seeder first SELECTs existing rows, builds a natural-key lookup (university: normalized name; department: (university_id, normalized name); professor: email first, then (department_id, normalized name), then (lab_id, normalized name)), and only inserts if no match is found — otherwise it diffs the payload and issues an UPDATE only if something changed, or a no-op "skipped" if identical. This makes re-running the seeder **idempotent**.

In-memory ID maps translate the JSON's human-readable string IDs (e.g. "university_01", "ian-reid") into real database UUIDs as each entity is seeded.

Error handling is per-record, not per-file. A record whose parent entity wasn't seeded is skipped with an error message recorded in EntityStats.messages rather than aborting the whole run.

Who runs it: backend/scripts/seed_data.py — invoked manually, not via any in-app endpoint or scheduled job.

Row-Level Security — Explicitly Not Used

NO DATABASE-LEVEL AUTHORIZATION BACKSTOP

The backend's Supabase client is constructed with the **service-role key** (backend/app/supabase_client.py:14, 19–22). The service-role key bypasses Postgres Row-Level Security entirely. Confirmed explicitly in the migration files:

"With the current service-role key access used in this application, the server enforces profile isolation at the application layer (profile_id checks)." — backend/migrations/20260909_outreach_drafts_gmail.sql:68–71

"No RLS policy is added: this table is only ever read/written by the backend's service-role Supabase client, never queried directly by a browser client." — backend/migrations/20260910_gmail_oauth_states.sql:22–24

Practical implication: authorization in this system is 100% an application-code guarantee (every service function filters by profile_id), not a database-enforced one. If a bug ever let one code path skip a profile_id filter, there is no RLS safety net underneath it.

6. Authentication vs. Authorization

Source: docs/architecture/auth-and-authorization.md

FindMyProfessor has **three distinct identity/permission systems** that are easy to conflate:

- **Application authentication** — "who is making this HTTP request to our backend?"
- **Profile/data isolation (authorization)** — "which student's data is this request allowed to touch?"
- **Gmail OAuth authorization** — "has this student granted us permission to send email as them, via Google?"

They are independent. A user can be authenticated (signed in with Google via Supabase) and still have no Gmail connection; a user can have a Gmail connection tied to a profile they no longer control the session for (in theory); and — critically — **for most of this app's history, "authentication" meant nothing more than a client-supplied header**, which is the gap the current code is in the middle of closing.

1. Application Authentication

What Identifies "the User" Today

Two signals travel on every API request, both attached by `frontend/src/services/api.js:23–51 (apiFetch)`:

- **X-Profile-Id header** — the application's own profile UUID, read from `localStorage.getItem('fmp_profile_id')` on the frontend.
- **Authorization: Bearer <token> header** — a live Supabase session access token, fetched fresh from `supabase.auth.getSession()` on every single API call.

CANDID DOCSTRING

The module docstring in `auth.py:3–33` is unusually candid about its own history: *"Every endpoint previously trusted a client-supplied X-Profile-Id header at face value, with no verification that the caller was actually the owner of that profile. Any client could set X-Profile-Id to any UUID and read/modify that profile's CVs, drafts, and Gmail connection."* (`auth.py:6–10`). That is the **original** authentication model, and it is still partially in effect today.

The Current (Partially Hardened) Model

`resolve_identity()` (`auth.py:156–196`) is the core dependency:

```
def resolve_identity(authorization, x_profile_id) -> Identity:
    user_id = get_authenticated_user_id(authorization) # verifies the Supabase JWT
    if not user_id:
        # No verifiable session at all — legacy path, unchanged behavior
        return Identity(profile_id=x_profile_id, user_id=None, authenticated=False)

    linked_id = _find_linked_profile_id(user_id)
    if linked_id:
        if x_profile_id and x_profile_id != linked_id:
            raise HTTPException(403, "X-Profile-Id does not match the authenticated account.")
        return Identity(profile_id=linked_id, user_id=user_id, authenticated=True)
    ...
```

`get_authenticated_user_id()` (`auth.py:53–73`) is real verification: it calls `supabase.auth.get_user(token)`, which round-trips to Supabase's own Auth server and validates the JWT signature/expiry there. This is **not** a locally-decoded, unverified JWT.

The rule in plain terms:

- If the request carries **no** Authorization header at all → the backend falls all the way back to trusting X-Profile-Id verbatim, exactly as the original (unsafe) design did. Explicitly called out as intentional transitional behavior (`auth.py:22–26`).
- If the request **does** carry a valid Supabase session → X-Profile-Id can no longer be used to reach a different account's profile. A mismatch is a hard 403. An X-Profile-Id for a profile that predates the user's first login gets auto-claimed once (`_claim_profile`, `auth.py:141–153`) via the `linked_user_id` column.

Why profiles.id Equals auth.users.id

`profiles.id` is engineered to literally be a Supabase Auth user id, not an app-generated UUID. Two creation paths exist (`backend/app/services/cv.py`):

- **User signs in with Google first, then uploads a CV** (the intended path going forward): `ensure_profile(profile_id=None, user_id=<real supabase user id>)` (`cv.py:71–110`) creates profiles with `id = user_id` directly.
- **User uploads a CV before ever signing in** (the original "no login required" design, still reachable): `_provision_auth_user()` (`cv.py:134–162`) calls `supabase.auth.admin.create_user(...)` to fabricate a **synthetic** `auth.users` row — a fake email like `student-<uuid>@local.findmyprofessor.invalid` with a random password the student never sees. The code comment is explicit: *"No login UI is added"* (`cv.py:135`). When that person later signs in with Google, `linked_user_id` connects their real Google identity to their pre-existing synthetic-identity profile, without migrating the profile's primary key.

HONEST ANSWER

"Who is this user?" — it depends which path they came through. If they signed in with Google first, `profiles.id` is a real, Google-verified identity. If they uploaded a CV first (still possible today — CV upload does not require a session), `profiles.id` initially points at a synthetic, no-one-ever-authenticates-as-this-directly `auth.users` row, and only becomes tied to a real person once/if they sign in with Google and the app auto-claims it.

What Is NOT Verified / the Remaining Gap

HEADLINE SECURITY CAVEAT

The module docstring states this outright (auth.py:29-32): "a caller presenting no Authorization header at all is still trusted on X-Profile-Id alone." Any client that simply omits the Authorization header can still access any profile by guessing/copying its UUID — the hardening only closes the gap for callers who *do* present a Supabase session.

Whether the frontend today always sends a session is one thing; whether some *other* client (a direct API caller, a script, curl) sends one is entirely out of the app's control.

This is the headline security caveat for this whole project — say it plainly in an interview rather than hiding it.

2. Authorization (Profile/Data Isolation)

There is no role system, no admin/student distinction, and no per-endpoint permission table. Authorization in this app means exactly one thing: **every query is filtered to profile_id = <the profile resolved by auth.py>**. This is enforced in application code, function by function — e.g. backend/app/outreach/service.py checks `draft.profile_id == profile_id` before returning or mutating any draft; backend/app/services/cv.py filters every `cv_versions` query by `profile_id`.

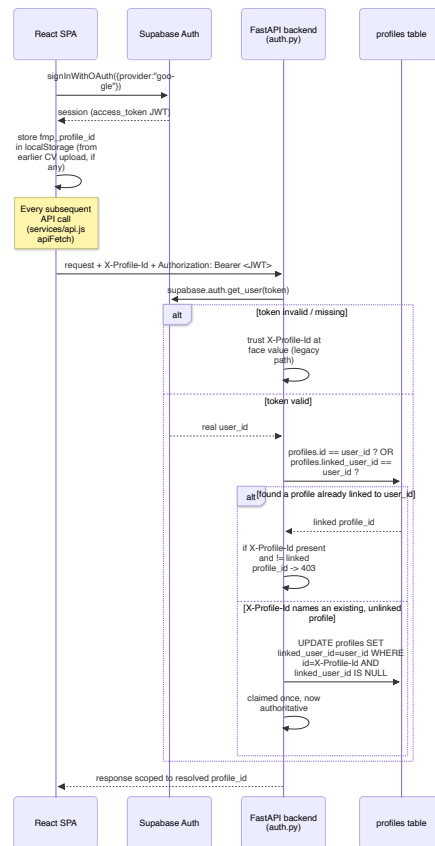
As covered in Chapter 5 (Row-Level Security), this is **not** backstopped by Postgres RLS, because the backend connects with the Supabase service-role key, which bypasses RLS entirely. Authorization is therefore a property of the backend's code being correct everywhere, not a property the database itself guarantees.

3. Gmail OAuth Authorization — A Third, Separate Thing

Being authenticated to the app is completely independent of having authorized Gmail. A user can use every feature of the app — CV upload, matching, drafting emails — without ever connecting Gmail. Connecting Gmail is a distinct, explicit, per-profile grant:

- **What external permission has the user granted?** Exactly one Google OAuth scope: `https://www.googleapis.com/auth/gmail.send` — send-only, confirmed as the literal scope string in backend/app/gmail/oauth.py:64. The app cannot read the user's inbox, list their contacts, or do anything with Gmail except send a message.
- This grant is stored as an encrypted access/refresh token pair in `gmail_connections`, one row per `profile_id` (`UNIQUE(profile_id, provider)`), not tied to the Supabase session at all — it persists independently of whether the user is currently "logged in."

Diagram — Application Authentication



Application authentication sequence — docs/architecture/auth-and-authorization.md §4

See Chapter 10 (Gmail OAuth) for the full Gmail sequence. The key point to internalize visually: that flow starts from an *already-resolved* `profile_id` (the connect endpoint requires one) and produces a completely separate artifact — an encrypted Google token pair in `gmail_connections` — that has nothing to do with the Supabase session token above.

Authentication vs. Gmail OAuth — Side-by-Side Comparison

Application Authentication

Who is this user?

A Supabase `auth.users.id`, verified via `supabase.auth.get_user()` — if a session token is presented at all.

What proves it?

A Supabase JWT, checked against Supabase's own Auth server (`auth.py:67`).

What can they access?

Only rows with matching `profile_id`, enforced in app code, not DB RLS.

What's the known gap?

No Authorization header at all → falls back to trusting X-Profile-Id unverified (`auth.py:29–32`).

Gmail OAuth Authorization

Who is this user?

The same `profile_id`, already resolved before the Gmail flow starts.

What proves it?

An encrypted OAuth token pair, checked against Google.

What can they access?

Only `gmail.send` — no read access to the user's real inbox.

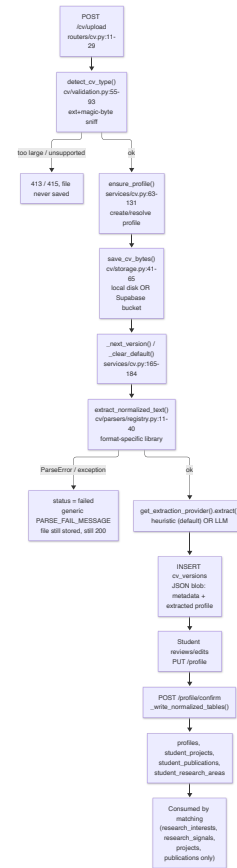
What's the known gap?

Token encryption key is required in production only via an `ENVIRONMENT` check, not the `RENDER` check used elsewhere (see Chapter 10).

7. CV Pipeline

Source: docs/architecture/cv-pipeline.md

Flow Overview



CV pipeline flow — docs/architecture/cv-pipeline.md §1 (required 'CV pipeline diagram')

Upload Endpoint

POST /cv/upload — backend/app/routers/cv.py:11–29, function `upload_cv()`. Uses `auth.resolve_identity` (not `require_profile_id`) specifically because this endpoint is allowed to *create* a brand-new profile if none exists yet. Reads the full file into memory and calls `cv_service.upload_and_parse()`. Two related endpoints: GET /cv/{cv_id} and POST /cv/{cv_id}/parse (re-run extraction without re-uploading).

Accepted File Types

Extensions: .pdf .docx .doc .txt .rtf .odt (cv/validation.py, SUPPORTED_EXTENSIONS, lines 9–16). Enforcement is **three layers deep**:

- Extension must be in the whitelist.
- `_sniff_format()` (validation.py:96–107) reads the actual file bytes: %PDF magic header for PDF, {\rtf for RTF, an OLE2 header for legacy .doc, ZIP-internal inspection for .docx/.odt, else falls back to a plain-text heuristic.
- The sniffed format must match the claimed extension (validation.py:78–82) — a .pdf file that is secretly a text file is rejected.

CONTENT SNIFFING IS AUTHORITATIVE

The browser-declared Content-Type is only a soft secondary signal and is ignored outright when it's the generic `application/octet-stream` (validation.py:83–91) — this is a meaningfully more robust validation approach than trusting the Content-Type header alone.

Size/Content Validation

MAX_CV_BYTES = 10 MB (validation.py:7). `validate_size()` (validation.py:46–52) rejects empty files (`<= 0` bytes) and anything over 10 MB, mapped to HTTP 413/415 respectively.

NO MALWARE SCANNING

No virus/malware scanning exists anywhere in the codebase (confirmed by search — zero hits for any scanning library or pattern).

Storage Location — Local Disk by Default

Two backends, selected by the SUPABASE_CV_BUCKET environment variable (storage.py:23):

- **Default (variable unset): local disk**, root backend/var/cv/uploads/, path pattern `{profile_id}/{cv_id}/{safe_filename}`. Path traversal is guarded via `resolve_storage_path()` (storage.py:68–77).
- **If SUPABASE_CV_BUCKET is set**: uploads to a private Supabase Storage bucket instead.

FLAGGED RISKY BY THE CODE'S OWN COMMENT

"Render's local disk does NOT persist across redeloys/restarts, so local disk alone is unsafe for production CV storage." (storage.py:11–22). Cross-referenced against every .env file present in this repo, **SUPABASE_CV_BUCKET is not documented and not set anywhere found** — meaning CV files (not their metadata — the cv_versions DB row survives fine) are at risk of being lost on every backend redeploy if the hosting platform is Render with an ephemeral filesystem. There is also **no file-deletion function anywhere** — uploaded CVs accumulate with no retention policy.

Text Extraction Per Format

Dispatch: backend/app/cv/parsers/registry.py:11–40.

Format	Library	Notes
PDF	<code>pypdf==6.18.0</code>	Handles empty-password-encrypted PDFs; raises <code>ParseError</code> on read/decrypt failure. No OCR — a scanned/image-only PDF with no text layer yields empty text and fails.
DOCX	<code>python-docx==1.2.0</code> (+lxml)	Standard OOXML parsing
DOC (legacy binary)	<code>olefile==0.47</code> + hand-rolled scraping	Not a real .doc parser — best-effort extraction of printable UTF-16LE/ASCII runs from the OLE2 container. Quality is noticeably lower than the other formats.
ODT	<code>odfpy==1.4.1</code>	Standard OpenDocument parsing
RTF	<code>striprtf==0.0.33</code>	Strips RTF control codes to plain text
TXT	<code>stdlib</code>	UTF-8 → UTF-8-sig → Latin-1 fallback chain

Structured Field Extraction — Heuristic vs. LLM

OFF BY DEFAULT

`get_extraction_provider()` (cv/extraction/factory.py:10–18) returns `HeuristicExtractionProvider` **unless the LLM_API_KEY environment variable is set**. This is an important, easy-to-miss fact: LLM-based CV extraction is off by default. The out-of-the-box behavior for every fresh deployment (including local dev with a blank .env.example) is the deterministic heuristic parser — zero cost, zero external API calls.

Heuristic path (cv/extraction/heuristic.py): splits the extracted text into sections using a fixed set of heading regexes (lines 23–29 — "Education", "Experience", "Skills", "Publications"), then applies per-section regexes:

- Identity: email regex, phone regex, a name heuristic (first non-empty line before any section heading).

- Education: degree/field/year/institution regexes — **only ever populates a single education record**, even if the CV lists multiple degrees.
- Skills: matched against a **hardcoded, closed catalog of ~31 skill names** (SKILL_CATALOG, heuristic.py:38-72) — a skill not on this list is never extracted, no matter how it's phrased.
- Experience/projects/publications/certifications: line-based extraction, capped at 8 entries each.
- research_interests / research_signals: matched against the live research_areas DB catalog — extraction quality for this field improves as more research-area labels get seeded, independent of code changes.

LLM path (cv/extraction/llm.py): there is **no LLM SDK dependency** in requirements.txt — the LLM call is a hand-rolled httpx POST to an OpenAI-compatible /chat/completions endpoint. Default LLM_BASE_URL=https://api.openai.com/v1, default LLM_MODEL=gpt-4o-mini.

NO FALLBACK BETWEEN PATHS

There is **no fallback from LLM to heuristic** if the LLM call fails — a failed LLM extraction fails the whole upload's parsing step, it does not silently degrade to the heuristic parser.

Extracted Schema Fields

ExtractedStudentProfile — cv/extraction/schema.py:65-74:

- identity — name, email, phone
- education[] — degree, field_of_study, institution, country, graduation_year, current_semester
- research_interests[] — explicit statements of interest
- research_signals[] — inferred-but-not-explicit research signal
- skills[] — name + category
- experience[] — role, organization, kind, description, start_year, end_year
- projects[] — title, description, technologies, research_relevance
- publications[] — title, venue, year, authors, publication_type
- certifications[] — name, issuer, year

HEURISTIC-ONLY GAPS

The heuristic provider never populates education.country, education.current_semester, or experience.organization — those fields exist in the schema but are LLM-only in practice under the default configuration.

Failure Handling

upload_and_parse() (services/cv.py:187-243) wraps the whole extract+parse step in a broad try/except:

- Unsupported format or oversized file → rejected before storage, HTTP 413/415, file never touches disk.
- File is a valid format but corrupt content → the file is still saved to storage, but cv_versions.description.parsing_status is set to "failed"; the HTTP response is still 200 OK (upload succeeded; parsing did not).
- Any other exception (including LLM HTTP/JSON/validation errors) is caught broadly and mapped to a single generic PARSE_FAIL_MESSAGE string — the real underlying error is logged server-side only.
- **There is no partial-success mode.** Extraction either fully succeeds or the entire CV is marked failed.
- confirm_profile() (services/cv.py:348-369) explicitly refuses to let a student confirm a profile whose active CV has parsing_status == "failed".

What Gets Persisted, and When

Every upload/reparse writes one cv_versions row containing the **entire** ExtractedStudentProfile as a JSON blob inside the description column, alongside file metadata and parsing status. This happens automatically on upload — no confirmation needed yet.

Only on **explicit POST /profile/confirm** does data get normalized into first-class columns/tables: profiles, student_projects, student_publications, and student_research_areas (write_normalized_tables, services/cv.py:372-433). Confirming **deletes and re-inserts** all three of those child tables in full each time — there's no incremental diffing.

What Matching Actually Consumes vs. What's Extracted but Unused

Extracted field	Consumed by matching?
research_interests	Yes — weight 0.55, the dominant scoring component
research_signals	Yes — weight 0.20
projects, publications (as evidence text)	Yes — weight 0.20 combined, via keyword-hit detection against matched areas
identity (name, email, phone)	No — used only for display/outreach personalization, not scoring
education	No
skills	No — explicitly excluded. A student's listed skills never contribute to a research-match score, confirmed by a dedicated test (test_skills_do_not_become_research_interests)
experience, certifications	No

Interestingly, `student_research_areas` (the normalized DB table written on profile confirm) also appears **not** to be read by the matching engine — scoring reads `research_interests/research_signals` directly from the CV's JSON blob, not from this normalized table. This looks like persisted-but-unused data; **NOT VERIFIED FROM CURRENT CODEBASE** whether some other code path reads it.

CV Versioning and Its Link to Outreach

`cv_versions.version_number/is_default` give a full, ordered history of every CV a student has uploaded. This matters for outreach: `outreach_drafts.cv_version_id` lets a student pin a **specific historical version** of their CV to a particular outreach draft — independent of whichever version is currently marked "default" — via POST `/outreach/drafts/{id}/attach-cv`. This is validated for existence again at send time.

Known Limitations (Evidenced by the Code)

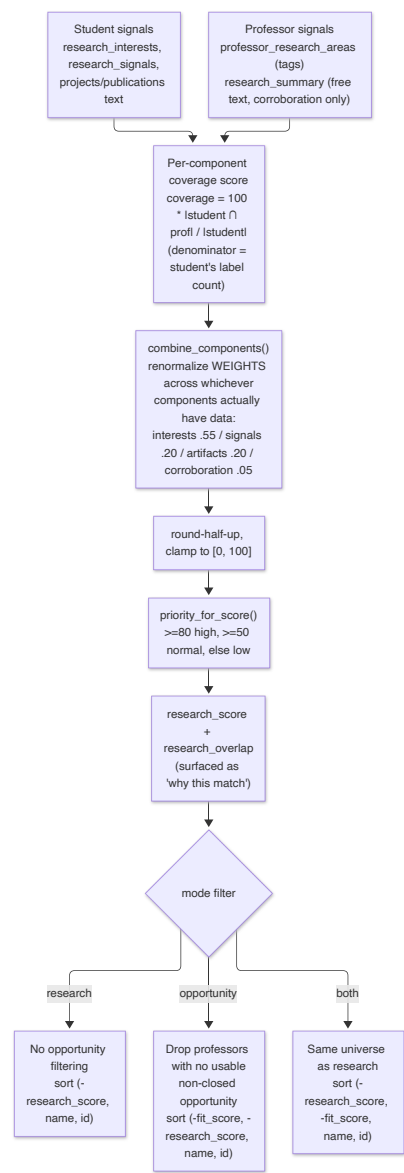
- No virus/malware scanning of uploaded files.
- No OCR — a scanned/image-only PDF produces an empty-text `ParseError` and a failed CV.
- Legacy `.doc` parsing is best-effort byte-scraping, not a real document parser.
- No fallback from LLM extraction failure to the heuristic parser — mutually exclusive per-request, selected once at the environment level.
- No file retention/deletion policy — uploads accumulate indefinitely.
- Default storage backend is local disk, unsafe on the inferred hosting platform (Render) unless `SUPABASE_CV_BUCKET` is explicitly configured — and it isn't, in any `.env` file present in this repo.
- Heuristic skill extraction is limited to a fixed ~31-item catalog; anything phrased differently is missed.
- Heuristic education extraction only ever captures one degree, even for CVs listing several.
- `student_projects/student_publications/student_research_areas` are written on confirm but appear unread elsewhere in the backend — likely dead data from the scoring engine's perspective (NOT VERIFIED as intentional).

8. Matching Engine

Source: docs/architecture/matching-engine.md

This is the core "product logic" of FindMyProfessor: given a student's confirmed profile, rank professors by research fit, optionally filtered/re-ranked by concrete funded opportunities. **It is entirely deterministic and rule-based — there is no ML model, no embeddings, no vector search, and no LLM anywhere in this engine.**

Matching Pipeline Diagram



Matching pipeline — diagram synthesized from the scoring pipeline described in docs/architecture/matching-engine.md §2–6 (not present as a diagram in the source; required 'matching pipeline diagram')

Endpoint

GET /matching/professors — backend/app/routers/matching.py:14–34, delegating to match_professors() in backend/app/services/matching.py:30.

Param	Type	Meaning
profile_id	via auth.require_profile_id	Whose confirmed CV profile to score against
mode	"research" "opportunity" "both" (default research)	See mode table below
limit	int, 1–100, default 25	Result cap after ranking
university_id	UUID, optional	Filters to one university

Param	Type	Meaning
<code>min_score</code>	int 0–100, default 0	Drop professors below this research score
<code>email_only</code>	bool, default false	Only professors with a listed email
<code>opportunity_type</code>	string, optional	internship/ra/masters/phd
<code>opportunity_status</code>	string, optional	open/upcoming/unknown/closed

Inputs

Student signals come from the confirmed CV profile (`_confirmed_student()`, `services/matching.py:111-129` — requires a confirmed CV or returns HTTP 400): `research_interests`, `research_signals`, and free text from `projects/publications` (title + description scanned for catalog-label hits). Notably, `skills` are never used here.

Professor signals are `professor_research_areas` (structured tags joined from the `research_areas` catalog) and `research_summary` (free text) — but the free text plays a strictly *corroborating* role, never a matching one (proven by `test_professor_summary_cannot_create_new_match`).

The Scoring Formula — `backend/app/matching/scoring.py`

```
WEIGHTS = {
    "interests": 0.55,
    "signals": 0.20,
    "artifacts": 0.20, # projects + publications
    "corroboration": 0.05, # research_summary text backing up a match
}
PRIORITY_HIGH = 80
PRIORITY_NORMAL = 50
```

Per-Component Coverage Score

```
coverage(component) = 0 if student has no labels in this component
                    = 100 * |student n prof| / |student| otherwise
```

DELIBERATE ANTI-BIAS CHOICE

The denominator is the **student's** label count, not the professor's. A professor listing many unrelated research areas does not dilute their score just for having a broad profile — proven by `test_area_count_bias_avoided`.

Corroboration is computed only if there is at least one matched area *and* the professor has a `research_summary`: $100 * (\text{matched areas that also textually appear in the summary}) / (\text{total matched areas})$.

Combining Components — the Renormalization That Matters Most

```
def combine_components(components: dict[str, float | None]) -> float:
    available = {k: v for k, v in components.items() if v is not None}
    total_weight = sum(WEIGHTS[k] for k in available)
    return sum(WEIGHTS[k] * v for k, v in available.items()) / total_weight
```

THE SINGLE MOST SURPRISING IMPLEMENTATION DETAIL

Weights are **renormalized** across whichever components actually have data, not applied against a fixed 100% denominator. A student with *only* explicit `research_interests` populated (no signals, no projects/publications, no professor summary to corroborate against) gets their interests component's weight treated as 100% of the final score, not capped at 55% — proven directly by `test_unavailable_components_renormalize: interests=100` alone yields a final score of 100, not 55. This means the "55/20/20/5" split is better described as *relative* priority weighting than a literal fixed formula.

Final score: round-half-up, then clamp to [0, 100]. **Priority label:** `score >= 80` → "high", `score >= 50` → "normal", else "low".

Text Normalization and the "Generic Terms" List

`normalize_label()` (`matching/normalize.py:9-13`) lowercases and collapses whitespace for every label comparison on both sides.

ONLY ONE HARDCODED GENERIC TERM

`GENERIC_RESEARCH_AREAS = frozenset({"computer science"})` (`normalize.py:6`) — this is a single hardcoded entry, not a broad stopwords list. It's stripped from both the student's and the professor's label sets before scoring, so matching purely on "Computer Science" contributes nothing to the score. If you were expecting a curated list of dozens of overly-generic terms — there isn't one; it's exactly this one string.

Research vs. Opportunity vs. Both Mode

All three modes compute the **identical** research score for every candidate professor. They differ only in filtering and sort order:

Mode	Filtering	Sort key
research	None (opportunity data not required)	(-research_score, name, id)

Mode	Filtering	Sort key
opportunity	Drops any professor with no usable, non-closed opportunity attached	(-fit_score, -research_score, name, id)
both	Same universe as research (opportunities attached but not filtering)	(-research_score, -fit_score, name, id)

Filters — Implementation Notes

- `min_score`: a strict `<` drop after scoring — a professor scoring exactly the threshold is kept, not dropped.
- `email_only`: a pure presence filter on professors.email at candidate-load time — proven not to influence the score itself.
- `university_id`: an inner-join filter through departments.university_id.
- `opportunity_type/opportunity_status`: filter the opportunity record set before it's attached to a professor, so these only affect which opportunity is considered "attached," not the research score.

Opportunity Scoring — backend/app/opportunities/scoring.py

```
STATUS_BASE = {"open": 100, "upcoming": 80, "unknown": 50}
```

closed opportunities are **filtered out entirely** before scoring even runs — they never appear as a candidate's "best opportunity."

Bonuses

- **+5** if `international_eligible` is `True` — applied unconditionally, regardless of the actual student's nationality (there is no nationality field checked against it).
- **+10** if `undergraduate_eligible` is `True` and the student is detected as clearly undergraduate via a regex-based check on their education text (`is_clearly_undergraduate`) — a graduate-degree regex match short-circuits this to `False` even for a student with a mixed/ambiguous education history.

Each opportunity's score is capped at `min(100, score)`.

BEST-SINGLE-OPPORTUNITY ONLY

`score_university_opportunities` picks only the **single best-scoring** non-closed opportunity for a given professor/lab/university — multiple good opportunities do not stack or add up (`test_multiple_opportunities_do_not_stack`). The result type's docstring is explicit: *"University-level context only. Never a professor-specific opportunity score."*

Opportunity ↔ Professor/Lab/University Relationship

`backend/migrations/20260908_opportunities_nullable_professor_lab.sql` exists specifically to drop `NOT NULL` on `professor_id/lab_id` "so university-wide (and lab-only) opportunities can be stored without fabricating a professor." Association resolution (`opportunities_for_professor`) is a **strict priority chain**: if the opportunity has a `professor_id`, match on that; else if it has a `lab_id`, match on that; else fall through to `university_id`. An opportunity is checked against exactly one tier, based on whichever foreign key is actually populated.

Worked Example (Real Numbers, Not Invented)

FROM TESTS/TEST_MATCHING_SCORING.PY:76-84

- Student `research_interests` = ["Computer Vision", "Medical AI", "Edge AI"]
- Professor `research_areas` = ["Computer Vision", "Medical AI", "Robotics"]
- No `research_signals`, no projects/publications, no professor `research_summary`

Step by step:

1. Intersection = {"Computer Vision", "Medical AI"} → 2 of the student's 3 interests.
2. `interest_score` = $100 * 2 / 3 = 66.667$
3. Only the interests component has data (weight 0.55); everything else is `None`.
4. `combine_components` renormalizes: $(0.55 / 0.55) * 66.667 = 66.667$ — the weight cancels out entirely because it's the only available component.
5. `clamp_score(66.667)` → round-half-up → **67**.
6. **Final research score = 67**, matching the test's exact assertion.
7. `priority_for_score(67)` → "normal" (below the 80 "high" threshold).
8. `research_overlap` = ["Computer Vision", "Medical AI"] is surfaced to the frontend as the human-readable "why this match" evidence.

Limitations (Evidenced by Code)

- Purely exact-label rule matching — no embeddings, no fuzzy/semantic similarity, no ML model of any kind. "Computer Vision" and "CV" would not match each other unless both are already normalized to the same catalog label upstream.
- The generic-terms exclusion list is exactly one entry ("computer science") — many other genuinely generic terms (e.g. "machine learning," "artificial intelligence") are not excluded and would inflate scores if broadly tagged.
- No feedback/learning loop — every score is a pure, deterministic function of the current data; running the same inputs twice always produces the same output.
- skills are entirely excluded from research matching.
- The international-student bonus ignores the student's actual nationality — fires purely off the opportunity's `international_eligible` flag.

- Opportunity fit scoring has no topical relevance component at all — a student's research match score and their opportunity fit score are computed completely independently and only combined by sort order in both/opportunity mode, never blended into one number.
- Best-opportunity aggregation discards information about how many good opportunities a professor/university actually has.
- No deadline-proximity or recency weighting anywhere in the scoring math.

Conceptual vs. Implementation — a Note for Interviews

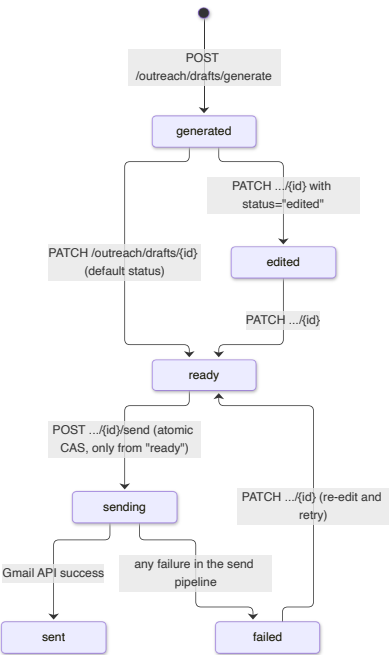
Conceptually, describe it as: "a weighted similarity score across four signal types — explicit interests, inferred signals, project/publication evidence, and free-text corroboration — normalized so a thinner CV isn't unfairly penalized for missing signal types." **Precisely**, the mechanism is: intersection-over-student-cardinality per component, weights renormalized across whatever components have data, rounded and clamped to 0–100. Both descriptions are accurate; the first is what you'd say to a non-technical interviewer, the second is what you'd say if asked to whiteboard it.

9. Email Generation

Source: docs/architecture/email-generation.md

Draft Lifecycle — State Machine

DraftStatus = "generated" | "edited" | "ready" | "sending" | "sent" | "failed"



Draft lifecycle state diagram — docs/architecture/email-generation.md §1 (required 'email generation flow')

Status	Set where	Trigger
generated	generate_draft() , service.py:107	POST /outreach/drafts/generate
edited	Only if the caller explicitly passes status="edited"	PATCH /outreach/drafts/{id}
ready	Default parameter of save_draft() and of SaveDraftRequest.status	PATCH /outreach/drafts/{id} (default case)
sending	atomically_set_sending() , a conditional UPDATE ... WHERE status='ready'	Only from inside send_draft(), only if current status is exactly ready
sent	service.py:332, after a successful send_message() call to Gmail	Terminal
failed	_mark_failed() (service.py:491-499), called from every failure branch of send_draft()	Any pipeline failure after sending was acquired

ENFORCEMENT GAP, STATED PLAINLY

save_draft() has **no state-machine guard** — it will overwrite status to whatever the caller sends, including jumping a fresh draft straight to "sent" via a raw PATCH. The only truly atomic, guarded transition is ready → sending. There is also a minor inconsistency: _validate_for_send() accepts both "ready" and "edited" as sendable, but atomically_set_sending() only succeeds from exactly "ready" — so an "edited" draft can pass validation and then still fail the atomic transition with a 409 ("Cannot send: draft status is 'edited'. Save as 'ready' first."). NOT VERIFIED FROM CURRENT CODEBASE whether this is intentional friction or an oversight.

Prepare/Generate — What Evidence Gets Gathered

POST /outreach/drafts/generate (routers/outreach.py:28-44) → generate_draft() (service.py:71-114). Body: professor_id, email_type ("research" | "research_opportunity"), optional opportunity_id. profile_id comes only from auth.require_profile_id.

Before any text is generated, the service assembles an **evidence bundle**, each piece independently verified against the database:

- _confirmed_student() (service.py:506-523) — loads the CV-extracted profile. **Hard-fails with HTTP 400 if the CV isn't uploaded, confirmed, or if its last parse failed.**
- _load_professor() (service.py:526-530) — 404 if the professor doesn't exist.
- _professor_context() (service.py:533-552) — builds a verified structure: name, title, email, university, department, lab, research areas, research summary — straight from the professors table join.
- _load_catalog() (service.py:555-559) — the shared research_areas catalog.
- _build_match_context() (service.py:562-595) — runs the **exact same deterministic score_professor()** used by the matching engine, producing research_overlap, shared_interest_areas, artifact_evidence, and corroborated_areas.
- _load_opportunity_context() (service.py:598-619) — optional; returns 400 if the selected opportunity is closed.

7. `_enforce_length()` (service.py:622-627) — truncates the generated body to 300 words, applied uniformly to both providers.

Provider Abstraction

backend/app/outreach/provider.py defines an abstract `EmailGenerationProvider` with one method, `generate(...)`. Two concrete implementations:

Deterministic Provider (Default, Zero Cost)

backend/app/outreach/deterministic.py, `DeterministicTemplateProvider` — pure string templating, **no network calls at all**.

- **Subject** (deterministic.py:156-166): picks the first overlap research area, rotates through 3 subject templates using a process-global counter.
- **Body** (deterministic.py:172-240): fixed structure — greeting ("Dear Professor {last_name},"), a rotating intro (3 templates), a research-opener paragraph plus a "best evidence" hook (prefers a publication over a project when both exist), a fit/ask paragraph, an optional opportunity paragraph, and a closing.

FABRICATION-PROOF BY CONSTRUCTION

Because every interpolated value traces directly to a verified field (`ProfessorContext`, `MatchContext`, or the CV-extracted profile), fabrication is structurally impossible on this path — there is nothing generative about it; it's `.format()` substitution.

LLM Provider (Optional, Requires an API Key)

backend/app/outreach/llm.py. Only active if `EMAIL_LLM_API_KEY` or `LLM_API_KEY` is set. Call config: `temperature=0.4`, `max_tokens=600`, `timeout=30.0`. Like the CV extractor, this is a hand-rolled httpx call to an OpenAI-compatible endpoint — no LLM SDK dependency in requirements.txt.

What Data Enters the LLM Prompt — and What Deliberately Doesn't

Enters the prompt (`_build_user_prompt()`, llm.py:48-97):

- Student name, a constructed degree phrase (degree + field + institution)
- The student's *verified* research interests
- The *verified shared* research areas (i.e. the actual overlap computed by `score_professor()`, not the student's raw wishlist)
- Student project/publication **titles**, but only the ones already tied to a matched area — not the student's entire artifact list
- Professor name, university, department, lab, research areas
- The requested email_type
- If an opportunity is selected: its title, type, status, and university — with an explicit prompt instruction that it's offered *by the university, not necessarily personally by the professor*

Deliberately excluded:

- The raw CV text/file bytes — never sent, only the pre-extracted, matching-verified structured fields chosen above
- The professor's raw free-text research_summary — only their structured research_areas list is sent, not their prose bio (reduces the model's opportunity to lift/misattribute claims from unstructured text)
- corroborated_areas — used only for post-hoc display, not fed into the prompt
- Professor email address, professor UUID, opportunity UUID, any auth tokens, or profile_id

SYSTEM PROMPT GUARDRAILS

(`_SYSTEM_PROMPT`, llm.py:30-45) instruct the model explicitly: use only the facts provided; never claim to have read a paper that wasn't listed; never claim the professor is actively recruiting unless that's stated; never invent topics, papers, labs, funding, or deadlines; omit missing information rather than guess it. The abstract `EmailGenerationProvider` class docstring states the identical contract.

Provider Selection and the Silent-Fallback Behavior

backend/app/outreach/factory.py:9-29: `EMAIL_LLM_API_KEY` takes priority over `LLM_API_KEY`; if neither is set, the deterministic provider is used — same default-safe pattern as CV extraction.

IMPORTANT FINDING

If the LLM call raises *any* exception, `LlmEmailProvider.generate()` (llm.py:117-142) **silently falls back to the full deterministic output** — and critically, the resulting draft's `generation_provider` field is recorded as "deterministic", which is **indistinguishable from the true default (no LLM configured at all) path**. There is no logging distinguishing "LLM was never tried" from "LLM was tried and failed."

Anti-Hallucination / Anti-Fabrication Measures — the Honest Picture

PREVENTATIVE ONLY, NEVER CORRECTIVE

There are exactly two mechanisms, both preventative (restricting inputs), and **zero** mechanisms that are corrective (checking outputs):

1. **Input restriction** — only verified, matching-scored fields are interpolated into the prompt at all. The model is never given the full CV or the professor's full bio to freely draw from.
2. **Prompt instructions** — the system prompt explicitly tells the model not to fabricate.

There is no post-generation validation anywhere in the code. `_parse_llm_response()` (llm.py:178-194) only splits the raw response into subject/body — it does not check that the professor's name, a research area, or an artifact title actually appears in the output. `_enforce_length()` is the only backend guardrail applied to LLM output, and it's a word-count truncation, not a fact-check.

Draft Storage

Table `outreach_drafts` (full DDL in `migrations/20260909_outreach_drafts_gmail.sql:11-32`): `id`, `profile_id` (FK → `profiles`, ON DELETE CASCADE), `professor_id` (plain UUID, no FK constraint), `email_type`, `subject`, `body`, `selected_opportunity_id` (FK → `opportunities`, nullable), `cv_version_id` (FK → `cv_versions`, nullable), `matched_research_areas/evidence_used` (JSONB), `generation_provider`, `status` (no DB CHECK constraint), `sent_at`, `gmail_message_id`, `gmail_thread_id`, `error_code`, `error_message`, `timestamps`.

Storage is via `backend/app/outreach/db_store.py`, which supports both a real Supabase-backed mode and an in-memory `_test_store` mode used by the test suite. Profile isolation is app-layer only (`service.py` checks `draft.profile_id == profile_id` before any read/write).

MINOR DISCREPANCY WORTH FLAGGING

`selected_opportunity_id` exists as both a model field and a DB column, but is **never actually set** inside `generate_draft()` despite an opportunity being loadable during generation — NOT VERIFIED as a bug vs. deliberate.

Edit Flow

PATCH `/outreach/drafts/{draft_id}` (`routers/outreach.py:47-60`) → `save_draft()` (`service.py:117-140`). Body: `subject`, `body`, `status` (default "ready"). A student can freely rewrite both subject and body, and set status to any of the six enum values in one call — there's no separate "confirm my edits" step distinct from the save itself.

CV Attachment Reference

`DraftRecord.cv_version_id` is set via **POST `/outreach/drafts/{id}/attach-cv`** → `attach_cv()` (`service.py:193-219`), which validates draft ownership, CV ownership, *and* that the underlying file still exists in storage before accepting the attachment. This is the field `_validate_for_send()` requires to be non-null before a send is allowed — see Chapter 11 (Gmail Send) for how the file is actually retrieved at send time.

Explicit Confirmation Requirement Before Send

Two independent, server-side gates (not just UI affordances):

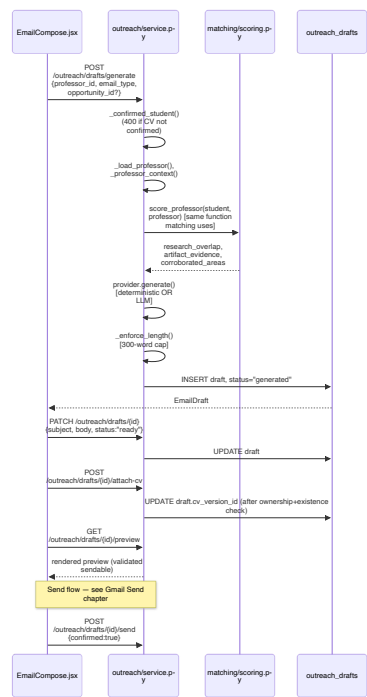
- Status gate:** draft status must be "ready" or "edited" (`_validate_for_send()`, `service.py:439-448`) — reached only through the edit/PATCH flow.
- Explicit confirmation flag:** `SendDraftRequest.confirmed: bool` is a **required Pydantic field with no default** (`schemas/outreach.py:79-81`) — omitting it is a validation error, not an implicit "no." `send_draft()` additionally re-checks it explicitly and returns 400 if falsy (`service.py:263-267`), so the guarantee doesn't rely on Pydantic alone.

Combined with the atomic ready → sending compare-and-swap DB transition (see Chapter 11), this makes sending a single, explicit, non-repeatable, user-confirmed action.

Failure Handling

Failure point	Behavior
CV not uploaded/confirmed/parse-failed	400 at generation time, no draft created
Professor not found	404
Selected opportunity is closed	400
LLM call fails during generation	Silently falls back to deterministic output — not a user-visible failure
DB write fails on <code>store.put()</code>	No explicit try/except around it in <code>generate_draft()</code> — would surface as an unhandled 500 (NOT VERIFIED whether a global handler catches this)
Any send-pipeline failure	Routed through <code>_mark_failed()</code> , which best-effort persists <code>status="failed"</code> + <code>error_code/error_message</code> , swallowing its own persistence errors with only a log line

Lifecycle Sequence Diagram



Email generation lifecycle sequence — docs/architecture/email-generation.md §12 (required 'email generation flow')

10. Gmail OAuth

Source: [docs/architecture/gmail-oauth.md](#)

This is a completely separate authorization system from application login (see Chapter 6). This chapter covers only how a student grants FindMyProfessor permission to send email through their own Gmail account.

1. Connect Endpoint

GET /gmail/connect — backend/app/routers/gmail.py:93–130. Accepts `profile_id` via X-Profile-Id header or query parameter. It cannot rely on a verified Supabase session here — this endpoint is reached by a **full-page browser navigation** (the user is redirected away to Google), which cannot carry a custom Authorization header the way a `fetch()` call can.

`return_to` (where to send the user back to in the SPA afterward) is sanitized against open-redirect abuse before use (gmail.py:74–86).

2. OAuth State Generation and Storage

State token: `secrets.token_urlsafe(32)` (gmail/oauth.py:99) — cryptographically random. Stored as a row in the `gmail_oauth_states` table: `token` (PK), `profile_id`, `return_to`, `expires_at`.

A REAL PRODUCTION LESSON BAKED INTO THE CODEBASE

This table replaced an in-memory Python dict. The migration file explains why, worth quoting directly: *"An in-memory store does not survive a Render restart/redeploy, and breaks entirely if the backend ever runs more than one worker/instance — a request handled by one process cannot see state created by another. This table makes state survive both."* (20260910_gmail_oauth_states.sql:6–9)

3. State Expiration

`_STATE_TTL_SECONDS = 600` — exactly 10 minutes (gmail/oauth.py:50).

4. Single-Use Enforcement

`consume_state()` (oauth.py:121–168) **deletes the row as part of reading it** — there's no separate "used" boolean to check and forget to enforce. Only the one request whose delete actually removes a row is allowed to proceed; a concurrent or replayed request finds nothing to delete and is rejected (explicit race-safety comment, oauth.py:149–154). Expiry is checked *after* the delete succeeds.

5. Authorization URL — Scope Confirmation

`authorization_url()` (oauth.py:187–199). The requested scope is **exactly one string**:

```
GMAIL_SEND_SCOPE = "https://www.googleapis.com/auth/gmail.send" # oauth.py:64
```

No read scope, no gmail.modify, no gmail.compose, no openid/profile/email scopes. `redirect_uri` comes from the required `GOOGLE_REDIRECT_URI` environment variable; `client_id` from `GOOGLE_CLIENT_ID`. `access_type=offline` and `prompt=consent` are both set, forcing Google to issue a `refresh_token` on every consent (not just the first time).

6. Callback Endpoint

GET /gmail/callback — backend/app/routers/gmail.py:137–224. Every distinct failure mode maps to a specific reason query param on the frontend error redirect:

Condition	reason
Google reports error= (user denied consent)	<code>denied</code>
Missing code or state	<code>missing_params</code>
State not found / already consumed / expired	<code>invalid_state</code>
Token exchange with Google fails	<code>exchange_failed</code>
Google returns no access_token	<code>no_token</code>
Writing the connection to the DB fails	<code>store_failed</code>

7. Token Encryption

backend/app/gmail/crypto.py. Algorithm: **Fernet** (from the cryptography package), keyed by the `GOOGLE_TOKEN_ENCRYPTION_KEY` environment variable.

Production behavior: if `GOOGLE_TOKEN_ENCRYPTION_KEY` is unset **and** `ENVIRONMENT` is "production"/"prod", the app hard-fails with a `RuntimeError` at the point of use — it refuses to encrypt tokens with a fallback key in a declared-production environment.

Development behavior: if unset in any other environment, the app generates an ephemeral key and persists it to a local file for the life of that dev instance, with a loud warning. This local key file is `var/gmail_dev.key` at the repo root, which is `.gitignore'd`.

A REAL INCONSISTENCY WORTH FLAGGING DIRECTLY

This production-readiness guard checks **only** the `ENVIRONMENT` variable. Elsewhere in the same codebase, `backend/app/routers/gmail.py:49–51` uses a **second, independent** signal — the `RENDER` environment variable that Render auto-injects into every deployed service — to detect "we are definitely running in a hosted environment" for a different production-hardening check (the `FRONTEND_URL` requirement), and the code comment there explains this was added specifically after a live incident where `ENVIRONMENT` was left unset on a real Render deployment. The token-encryption guard in `crypto.py` was **not** updated to use the same `RENDER` check. Practically: if a Render deployment forgets to set `ENVIRONMENT=production` (exactly the scenario that already happened once), Gmail tokens would be silently encrypted with an ephemeral, non-persistent key instead of hard-failing. NOT VERIFIED FROM CURRENT CODEBASE whether this is accepted risk or simply not yet backported — it's a legitimate, code-grounded "next thing I'd fix."

8. Token Storage

Table `gmail_connections` (`migrations/20260909_outreach_drafts_gmail.sql:45-62`): `id`, `profile_id` (FK → `profiles`, cascade delete), `provider` (always "google" today), `provider_account_email` (plaintext — safe, never a token), `access_token_encrypted`, `refresh_token_encrypted`, `token_expires_at`, `scopes`, `revoked_at`, `UNIQUE(profile_id, provider)`. Written via `upsert_connection()` (`gmail/service.py:64-98`) — a fetch-then-update-or-insert pattern rather than a native atomic upsert; whether a genuinely concurrent first-time connect race is safe against the unique constraint is NOT VERIFIED FROM CURRENT CODEBASE.

No RLS policy exists for this table either — same app-layer-only isolation pattern as everywhere else.

9. Token Refresh

`get_valid_access_token()` (`gmail/service.py:133-201`) is called lazily, right before every send — there's no background refresh job. It refreshes if the stored token is within 60 seconds of expiry. If Google rejects the refresh attempt (400/401 — `revoked/invalid`), the connection is marked unusable: `_mark_connection_unusable()` sets `revoked_at`, and a 401 with a "please reconnect" message is raised. Transient network errors or Google 5xx responses do **not** revoke the connection — only an explicit rejection from Google does. Errors are logged with only the HTTP status and Google's own error fields — the actual token values are never logged.

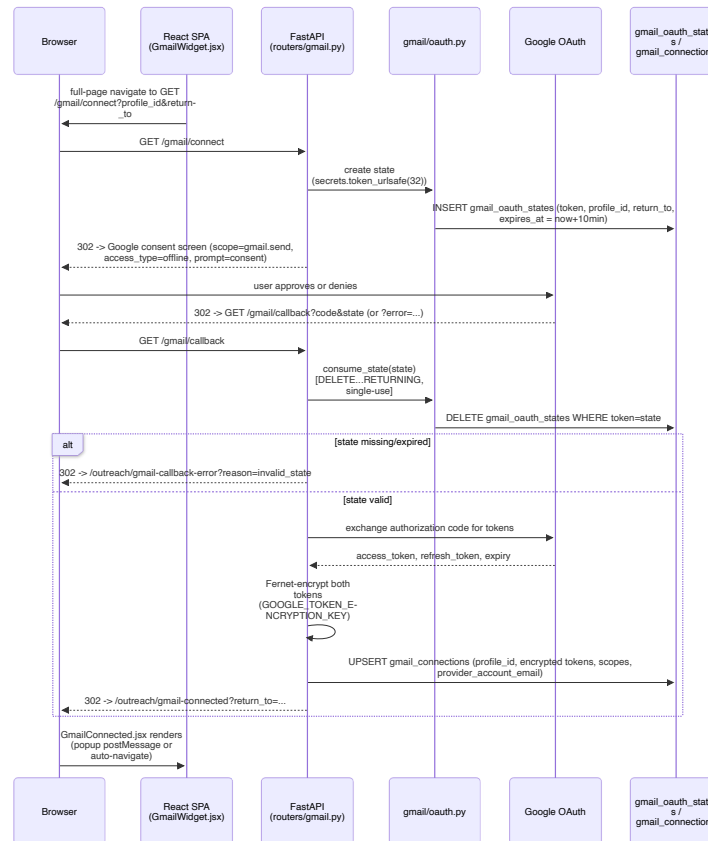
10. Frontend Callback Handling

The backend redirects the browser directly to React SPA routes — `/outreach/gmail-connected` and `/outreach/gmail-callback-error` — **not** to any server-rendered page. `GmailConnected.jsx` supports two integration patterns: a popup flow (posts a message to the opener window and closes itself) or a direct `navigate-back-to` return_to/a pending intent stashed in `localStorage`. `GmailError.jsx` maps the `?reason=` query param to user-facing copy.

DEAD LEGACY FILES

The static files `backend/app/static/gmail_connected.html` and `gmail_error.html` still exist on disk but are dead. They're mounted under `/static` as raw static assets, but nothing in the live OAuth redirect logic references them.

11. Full Sequence Diagram



Gmail OAuth full sequence — docs/architecture/gmail-oauth.md §11 (required 'Gmail OAuth sequence diagram')

11. Gmail Send Flow & CV Attachment

Source: <docs/architecture/gmail-send.md>

1. Send Endpoint and Validation

POST /outreach/drafts/{draft_id}/send — backend/app/routers/outreach.py:129–148 → send_draft() (service.py:246–346). Body: SendDraftRequest{confirmed: bool} — confirmed has **no default**, so omitting it is a validation error, not an implicit "no."

Before anything is sent, _validate_for_send() (service.py:415–488) checks, in order:

1. Draft ownership (draft.profile_id == profile_id)
2. Subject and body are both non-empty
3. Status is "ready" or "edited" (see the status-vs-CAS inconsistency noted in Chapter 9)
4. Professor exists and has a regex-validated email address
5. cv_version_id is set, and that CV version still belongs to this profile and still exists in storage
6. A live Gmail connection exists for this profile

2. Explicit Confirmation Requirement, Enforced Twice

- confirmed is a required schema field (not just a UI checkbox) — schemas/outreach.py:79–81.
- send_draft() **re-checks it explicitly in code** and returns HTTP 400 if it's falsy (service.py:263–267) — so the guarantee doesn't rely on Pydantic validation alone; even a client that bypasses schema validation still hits the same code-level check.

3. Atomic Status Transition — Preventing Double-Send

atomically_set_sending() (outreach/db_store.py:180–201) issues a single conditional update:

```
UPDATE outreach_drafts
SET status = 'sending'
WHERE id = ? AND profile_id = ? AND status = 'ready'
```

CLASSIC COMPARE-AND-SWAP

If two send requests race for the same draft (e.g. a double-click, or a retried request), only one can successfully flip ready → sending; the loser's UPDATE affects zero rows and the request is rejected with 409 Conflict. This is what makes "explicit confirmation" also mean "exactly-once," not just "asked nicely once."

4. CV Attachment

read_cv_bytes(storage_path) (cv/storage.py:79–89) retrieves the file server-side — either from a private Supabase Storage bucket via the backend's own service-role client, or from local disk with path-traversal guards.

THE STORAGE PATH NEVER APPEARS IN ANY API RESPONSE SCHEMA

The frontend only ever holds a cv_version_id, and the backend resolves that id to a file path internally at send time. This is the mechanism that keeps CV files private without needing a public URL.

5. MIME Construction

backend/app/gmail/client.py:37–82 — built with the Python standard library's email.mime (MIMEMultipart, MIMEText, MIMEApplication), not a third-party mail library. The message body is plaintext only (no HTML variant). The From header is omitted when unknown; Gmail fills it in itself since the API call authenticates as user_id='me'. The finished MIME message is base64url-encoded into {"raw": ...}, the exact format the Gmail API's messages.send expects.

6. Gmail API Call

googleapiclient.discovery.build("gmail", "v1", ...), then:

```
service.users().messages().send(user_id="me", body=mime_message).execute()
```

Confirmed as the literal call — client.py:99–107. The response's id and thread_id fields are captured and stored as gmail_message_id/gmail_thread_id on the draft. Neither value is ever written to logs.

7. Error Handling

Source	Handling
No valid Gmail token available	401, "please reconnect"
Token refresh rejected by Google (400/401)	Connection marked revoked_at (see Chapter 10 §9); 401 to caller

Source	Handling
Gmail API returns 400/401/403 on the send call itself	Wrapped as GmailApiError, surfaced as a generic, safe 502 to the client (no raw Google error text leaked), draft marked status="failed" with the extracted error code
MIME construction fails	Draft marked failed
CV file missing from storage at send time	Draft marked failed

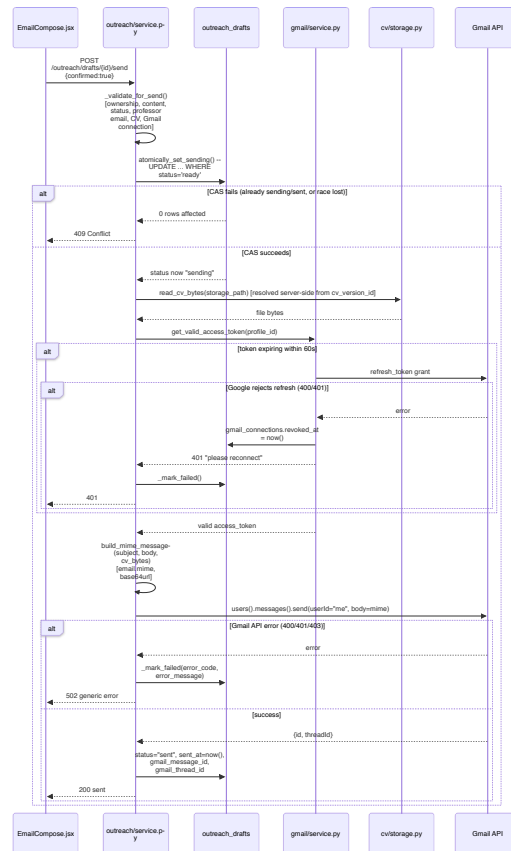
Every failure path funnels through `_mark_failed()` (service.py:491-499), which leaves the draft in a terminal failed state — **a draft can never get stuck in sending** after an error; it either reaches sent or failed, never hangs in between.

8. Why the System Does Not Auto-Send or Auto-Follow-Up

CONFIRMED: THERE IS NO AUTOMATION OF ANY KIND

A repository-wide search for any cron, scheduler, Celery, APScheduler, or background-task pattern across backend/app returns nothing. Every send is a single, synchronous HTTP request, gated by the explicit `confirmed:true` flag, initiated only by direct user action in the browser. There is no code path that could send an email without a human clicking "Send" at that exact moment, and no code path that revisits a sent draft later to send a follow-up. This is a deliberate safety property worth stating explicitly in an interview: the absence of automation isn't an oversight, it's the simplest possible way to guarantee a student is never surprised by an email going out on their behalf.

9. Full Sequence Diagram



Gmail send full sequence — docs/architecture/gmail-send.md §9 (required 'Gmail send sequence diagram')

12. Frontend Architecture

Source: docs/architecture/frontend.md

1. Stack

React 18.3.1, react-router-dom 6.26.0, @supabase/supabase-js 2.45.0, built with Vite 5.4, tested with Vitest 2.1.9 + Testing Library. No CSS framework — hand-written CSS + inline styles. frontend/vite.config.js proxies backend path prefixes (/api, /profile, /cv, /matching, /professors, /universities, /outreach/drafts, /outreach/cv-versions, /gmail/*) to http://localhost:8000 in dev.

2. Route Table

Defined in frontend/src/App.jsx:20–35 (BrowserRouter).

Path	Component	Gated?
/	Home	No
/team	Team	No
/login	Login	No
/onboarding	Onboarding	Soft — needs fmp_profile_id in localStorage
/matches	Matches	Soft — shows EmptyState if no profileId
/matches/:id	ProfessorDetail	No explicit gate
/outreach/compose/:professorId	EmailCompose	Soft — needs profileId + confirmed profile
/outreach/history	OutreachHistory	Soft — needs profileId
/outreach/gmail-connected	GmailConnected	No
/outreach/gmail-callback-error, /gmail-error	GmailError	No
/privacy	Privacy	No
/terms	Terms	No
*	redirect to /	N/A

IMPORTANT FINDING: THERE IS NO HARD ROUTE GUARD ANYWHERE

No page uses `useAuth()` to redirect an unauthenticated visitor to `/login`. "Gating" is entirely a soft, presence-based check on a `localStorage` value that has nothing to do with whether a Supabase session exists — a user could, in principle, clear their Supabase session but keep `fmp_profile_id` in `localStorage` and continue using pages that only check for the latter. `useAuth()` is otherwise consumed only by `AppShell` to render the account menu.

3. Authentication on the Frontend

Google OAuth via Supabase Auth, exclusively — no email/password, no magic link.

```
// frontend/src/pages/Login.jsx:64–67
supabase.auth.signInWithOAuth({
  provider: 'google',
  options: { redirectTo: (VITE_SITE_URL || 'http://localhost:5173') + '/login' }
})
```

`Login.jsx:41–58` checks `getSession()` on mount and subscribes to `onAuthStateChange`, navigating to `/onboarding` on `SIGNED_IN`. `useAuth.js` is a general-purpose session hook wrapping the same two calls; its `signOut()` also clears `fmp_profile_id` from `localStorage`.

MINOR FINDING

The Supabase client (`frontend/src/lib/supabase.js:3–6`) is constructed with a **hardcoded URL and anon key directly in source**, not read from `import.meta.env` — unusual, since the Supabase anon key is meant to be safely public, but it does mean the frontend build doesn't actually need Supabase-related env vars at all; only `VITE_API_URL` and `VITE_SITE_URL` are read from the environment anywhere in the frontend.

4. How Identity Reaches the Backend

Every backend call goes through one helper, and this is the single most important fact about the frontend/backend boundary:

```
// frontend/src/services/api.js:23-51 (apiFetch)
export async function apiFetch(url, options = {}) {
  const profileId = getProfileId(); // localStorage['fmp_profile_id']
  const headers = { ...options.headers };
  if (profileId) headers['X-Profile-Id'] = profileId;

  const { data } = await supabase.auth.getSession();
  const token = data?.session?.access_token;
  if (token) headers['Authorization'] = `Bearer ${token}`;

  return fetch(apiUrl(url), { ...options, headers });
}
```

Both headers are attached on every single request, unconditionally when available. The Supabase session token is re-fetched fresh from `getSession()` on every call (not cached client-side), so it's always current. See Chapter 6 for exactly how the backend reconciles these two signals.

5. Gmail Connection State

`useGmailStatus.js` does a **one-time fetch on mount** — no polling, no shared context/cache. `AppShell`, `GmailWidget`, and `EmailCompose` each mount their own independent instance of this hook, so the connect status is fetched multiple times per page load. Freshness after a connect/disconnect action depends on an explicit `refetch()` call or a full page remount.

6. Page-by-Page Walkthrough

- **Onboarding.jsx** — 3-step CV flow: (1) drag/drop upload → `uploadCV()` → `POST /cv/upload` as `FormData`, stores the returned `profile_id`; (2) review/edit the extracted profile → `updateProfile()` → `PUT /profile`; (3) confirm → `confirmProfile()` → `POST /profile/confirm`.
- **Matches.jsx** — filter UI (university, min score, email-only, opportunity type/status, mode) maps directly to `fetchMatches()` query params against `GET /matching/professors`; a separate client-side text search further narrows the already-fetched result set without another backend call.
- **ProfessorDetail.jsx** — fetches `GET /professors/{id}` for the detail view, plus a non-blocking `GET /matching/professors?limit=100` purely to locate this professor's score/evidence object client-side — there is no single-professor match endpoint on the backend.
- **EmailCompose.jsx** (966 lines, the most complex page) — on mount: restores a pending draft (from an OAuth-return intent, or `listDrafts()`), fetches the student profile, the target professor, and (non-blocking) the match list; once a draft exists, also lists CV versions. Generate → `POST /outreach/drafts/generate`. Save → `PATCH /outreach/drafts/{id}`. Attach CV → `POST /outreach/drafts/{id}/attach-cv`. Connect Gmail → auto-saves the draft, then does a full-page redirect to `GET /gmail/connect`. Preview → auto-saves if needed, then `GET /outreach/drafts/{id}/preview`. Send → `POST /outreach/drafts/{id}/send`.
- **OutreachHistory.jsx** — lists past drafts/sends for the profile.
- **GmailConnected.jsx / GmailError.jsx** — receive the backend's OAuth redirect; see Chapter 10 §10.

7. API-Call Inventory (Cross-Referenced Against Backend Routers)

Every frontend service call was cross-checked path-for-path against `backend/app/routers/*.py` and the router registration in `main.py` — **no orphaned frontend calls were found**. One asymmetry worth noting: `backend/app/routers/departments.py` and `labs.py` exist and are registered, but no frontend service calls their endpoints directly — the frontend reaches `labs/departments` only indirectly, through the `professor/university` endpoints. NOT VERIFIED whether this is simply unused-for-now surface area or reserved for a future drill-down UI. See Chapter 17 (API Map) for the full endpoint reference.

8. Frontend Build in Production

Root `.gitignore` excludes `backend/app/static/react/`, implying the intended production setup copies the frontend's Vite build output into that path for FastAPI to serve directly — but no CI/build script performing that copy exists in this repository, so the actual production build/deploy mechanism is **NOT VERIFIED FROM CURRENT CODEBASE**. See Chapter 14 (Deployment).

13. Backend Architecture (FastAPI)

Source: [docs/architecture/backend.md](#)

1. App Wiring

`backend/app/main.py` is the entire application entrypoint (100 lines). It:

- Builds an explicit CORS origin allow-list (`_cors_allowed_origins()`, `main.py:22-42`) — never a wildcard. The default set is `{"https://findmyprofessor.online", "http://localhost:5173", "http://127.0.0.1:5173"}`, extendable via `FRONTEND_URL` and `CORS_EXTRA_ORIGINS`. Because identity is header-based, not cookie-based, a wildcard origin combined with `allow_credentials=True` would let *any* website's JavaScript make authenticated-looking cross-origin calls against this API.
- Registers 9 routers (`main.py:59-67`): universities, departments, labs, professors, cv, profile, matching, outreach, gmail.
- Mounts `/static` for the legacy static HTML directory — see Chapter 14 §6.
- Exposes `GET / (API identity/links)` and `GET /health` (pings universities table via Supabase, returns 503 if unreachable).
- A comment block (`main.py:73-77`) states outright that legacy server-rendered HTML pages have been removed from FastAPI's routing and the React SPA now owns all product pages.

2. Request Lifecycle



Request lifecycle — [docs/architecture/backend.md §2](#) (part of the required "frontend/backend architecture")

Concretely, for a typical endpoint (e.g. `GET /matching/professors`):

- Router** (`backend/app/routers/matching.py`) declares the path, query params, and a `response_model`.
- Dependency injection** resolves `profile_id` via `app.auth.require_profile_id` — this is where identity/authorization happens, before any business logic runs.
- Service function** (`backend/app/services/matching.py`) does the actual work — loading data, calling into `app/matching/scoring.py` and `app/opportunities/scoring.py`, filtering, sorting.
- Database access** goes through `app.services.query.execute()` (`backend/app/services/query.py`) — a thin wrapper around every Supabase call that maps Postgres errors to safe, generic HTTP errors and logs the real error server-side.
- Response** is validated against a Pydantic model in `app/schemas/matching.py` before being returned.

3. Directory Responsibilities

Directory	Responsibility
<code>app/routers/</code>	HTTP surface only — path/method/param declarations, delegates immediately to a service
<code>app/schemas/</code>	Pydantic request/response models — the API's public contract
<code>app/services/</code>	Business logic for the "catalog" domain — thin orchestration over Supabase queries
<code>app/matching/</code>	Pure scoring logic for professor research-fit (<code>scoring.py</code> , <code>evidence.py</code> , <code>normalize.py</code> , <code>models.py</code>) — no I/O
<code>app/opportunities/</code>	Pure scoring logic for opportunity fit — no I/O
<code>app/cv/</code>	CV subsystem: <code>parsers/</code> , <code>extraction/</code> , <code>storage.py</code> , <code>validation.py</code>
<code>app/outreach/</code>	Email draft lifecycle: <code>service.py</code> , <code>deterministic.py</code> , <code>llm.py</code> , <code>db_store.py</code> , <code>store.py</code> , <code>models.py</code>
<code>app/gmail/</code>	Gmail integration: <code>oauth.py</code> , <code>client.py</code> , <code>crypto.py</code> , <code>service.py</code>
<code>app/ingestion/</code>	One-time/manual data-loading pipeline from <code>/data/*.json</code> into Supabase — not part of request handling
<code>app/auth.py</code>	The one shared identity-resolution module every router depends on
<code>app/supabase_client.py</code>	Single shared Supabase client instance, constructed with the service-role key
<code>app/static/</code>	Legacy, no-longer-routed HTML pages

4. A Cross-Cutting Pattern: `services/query.py`

Every Supabase call in the codebase is expected to pass through `execute()` (`backend/app/services/query.py:9-30`), which:

- Catches `postgest.exceptions.APIError` and maps it to a **generic, safe** client-facing message ("Database query failed.", 503; or "Invalid ID format.", 400 for a malformed UUID) — the real Postgres error code/message/hint is logged server-side only, never leaked to the client.
- Provides `is_missing_column_error()` (`query.py:51-68`) — lets a caller detect "this specific optional column doesn't exist yet" (e.g. `linked_user_id` before its migration is applied) and degrade gracefully instead of failing the whole request. Actively used in `app/auth.py` and `app/services/cv.py`.

5. Dependencies (from backend/requirements.txt)

Category	Packages
Web/ASGI	fastapi 0.141.1, starlette 1.6.0, uvicorn 0.52.4, python-multipart
Validation	pydantic 2.13.5
HTTP client	httpx (also used for hand-rolled LLM calls)
Supabase	supabase 2.31.0, supabase-auth, supabase-functions, postgrest, storage3, realtime
Crypto/auth	cryptography 50.0.1 (Fernet token encryption), PyJWT
Google/Gmail	google-auth, google-auth-oauthlib, google-api-python-client
CV parsing	pypdf, python-docx (+lxml), striptrf, odfpy, defusedxml, olefile
LLM	No SDK — both the CV extractor and the email generator make raw httpx calls to an OpenAI-compatible endpoint
Testing	pytest 9.1.1

There is no ORM — every database interaction goes through the supabase-py query builder directly (`.table(...).select(...).eq(...).execute()`), not SQLAlchemy or similar.

14. Deployment Architecture

Source: docs/architecture/deployment.md

1. No Deployment Manifest Exists in This Repository

HOSTING CONFIGURATION IS NOT VERSION-CONTROLLED

An exhaustive search (excluding node_modules, .git, venv) found **zero** deployment configuration files: no Dockerfile, docker-compose.yml, render.yaml, railway.json/.toml, vercel.json, netlify.toml, Procfile, fly.toml, app.json, or .github/workflows/*.yml. Everything below about the actual production environment is inferred from code comments, environment-variable checks, and non-secret URL values found in .env files — not from an authoritative manifest. Treat it as strong circumstantial evidence, not confirmed fact.

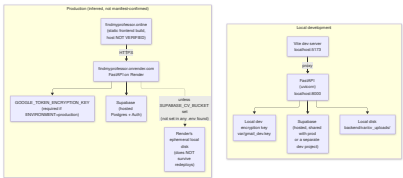
2. Evidence Pointing to the Hosting Platform

Evidence	Location	What it implies
Comment naming the platform directly	backend/app/main.py:14-18: "a misconfigured/missing Render env var can't silently break the deployed SPA"	Backend is deployed on Render
A functional check on Render's own auto-injected env var	backend/app/routers/gmail.py:51, checking RENDER (not in .env.example — Render sets this automatically on every deployed service)	Strong confirmation — this is a runtime behavioral dependency on Render, not just a comment
Production API URL	frontend/.env.production:2: VITE_API_URL=https://findmyprofessor.onrender.com	Confirms the backend's actual production domain is an .onrender.com subdomain
Production frontend domain	frontend/.env.production:1 and main.py:18: https://findmyprofessor.online	Confirms the product's custom domain; the frontend's actual hosting platform is not confirmed — could be static hosting on Render, Vercel, Netlify, or elsewhere

NOT VERIFIED FROM CURRENT CODEBASE

The exact frontend hosting provider, whether CI/CD exists at all, how a deploy is actually triggered, and the specific backend web worker/process configuration.

3. Local Development vs. Production



Local vs. production deployment — docs/architecture/deployment.md §3 (required 'deployment architecture' diagram)

4. Environment Variable Inventory

Variable	Purpose	Required in prod?	Notes
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY	Backend DB/auth client	Yes — app fails to start without these	Service-role key bypasses RLS entirely; see Chapter 5 §RLS
SUPABASE_CV_BUCKET	Durable CV storage backend	No (optional)	Undocumented in .env.example, not set in any .env file found in this repo. Without it, CV files default to local disk
LLM_API_KEY, LLM_BASE_URL, LLM_MODEL	CV extraction LLM provider	No — blank means the deterministic/heuristic path is used	See Chapter 7 §7
EMAIL_LLM_API_KEY, EMAIL_LLM_BASE_URL, EMAIL_LLM_MODEL	Email generation LLM provider, overrides LLM_* if set	No — same deterministic-by-default pattern	See Chapter 9 §5
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI	Gmail OAuth	Yes, for the Gmail feature to function	GOOGLE_REDIRECT_URI must exactly match what's registered in Google Cloud Console
GOOGLE_TOKEN_ENCRYPTION_KEY	Fernet key for encrypting stored Gmail tokens	Hard-required only when ENVIRONMENT is production/prod	See Chapter 10 §7 for the flagged inconsistency with the RENDER check used elsewhere
ENVIRONMENT	Switches on stricter production checks	Should be production in prod	Default development; not present as a key in the root .env or backend/.env files inspected

Variable	Purpose	Required in prod?	Notes
RENDER	Render's own auto-injected marker	N/A — set automatically by the platform	Checked directly in backend/app/routers/gmail.py:51 as a second, independent "are we hosted" signal
FRONTEND_URL, CORS_EXTRA_ORIGINS	CORS allow-list additions	Effectively required in a hosted environment (raises RuntimeError if hosted and blank)	main.py:34-42
VITE_API_URL, VITE_SITE_URL (frontend)	Backend API base URL and canonical site URL	Yes for a correct prod build	Dev: localhost:8000 / localhost:5173. Prod: findmyprofessor.onrender.com / findmyprofessor.online

5. CV Storage: Local Disk by Default, Flagged Unsafe by the Code Itself

AS CURRENTLY CONFIGURED, A BACKEND REDEPLOY ON RENDER WOULD LOSE PREVIOUSLY UPLOADED CV FILES

backend/app/cv/storage.py:11-22 states directly that Render's local disk does not persist across redeployments/restarts, so local-disk-only storage is unsafe in production — yet SUPABASE_CV_BUCKET, the escape hatch that switches to durable Supabase Storage, is absent from every .env file present in this repo. While their database metadata (cv_versions rows) survives untouched.

6. Legacy Static HTML Pages

backend/app/static/ contains 11 HTML files — login.html, matches.html, professor_match.html, outreach_history.html, cv_onboarding.html, email_draft.html, privacy.html, terms.html, index.html, gmail_connected.html, gmail_error.html — evidently an earlier, server-rendered version of the product, predating the React SPA.

CONFIRMED DEAD AS APPLICATION PAGES

main.py:70 mounts StaticFiles at /static only (serving raw files, not routing pages), and main.py:75-77 states directly: *"Legacy HTML pages... have been removed from FastAPI. The React frontend... renders them."* A repository-wide search for any FileResponse/send_file/.html reference inside backend/app returns zero hits — no router serves any of these files as an actual page route. They remain fetchable byte-for-byte at /static/<name>.html, but nothing in the live application links to or navigates through them.

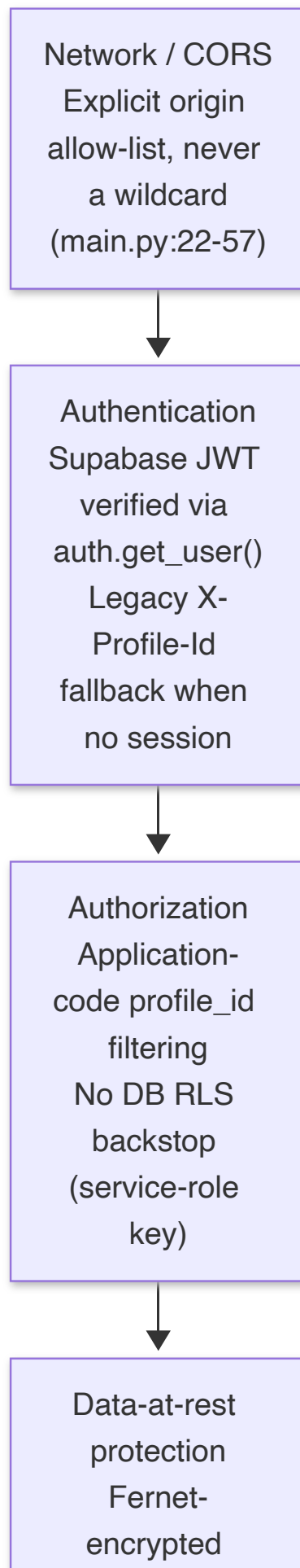
7. Root package.json

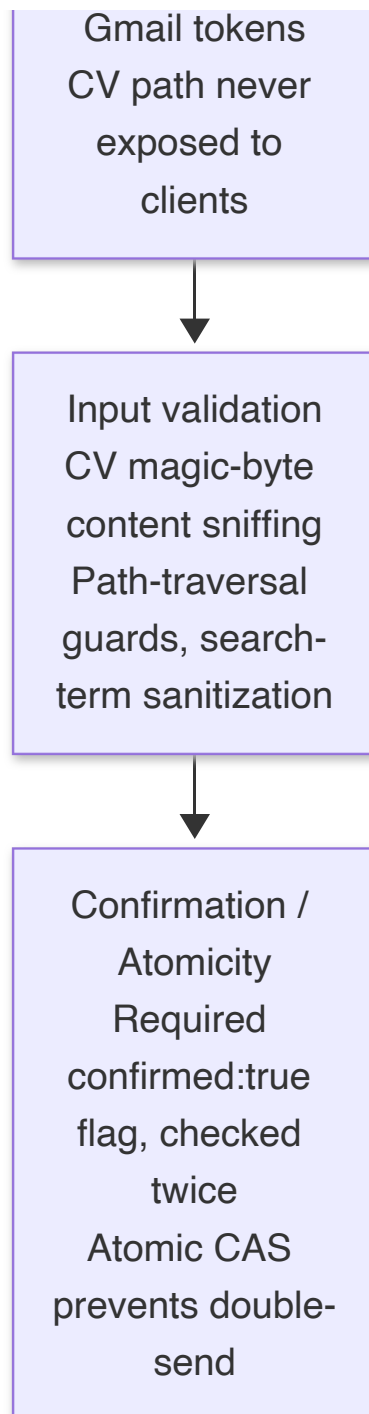
Confirmed to be a thin convenience wrapper only — two scripts (test:frontend, test:backend), no dependencies of its own. It is **not** a monorepo build tool; the frontend and backend are built and (presumably) deployed independently.

15. Security Architecture

Source: docs/architecture/security.md — states only what is actually implemented, verified against code; no invented vulnerabilities, no hidden mitigations.

Security Layers Diagram





Security architecture — diagram synthesized from the mechanisms table in docs/architecture/security.md §1 (not an original source diagram; required 'security architecture' diagram)

1. Mechanisms Actually Implemented

Supabase JWT verification

backend/app/auth.py:53-73
supabase.auth.get_user(token) — a real round-trip check against Supabase's Auth server, not a locally-decoded/unverified JWT

Explicit CORS allow-list

backend/app/main.py:22-57
No wildcard origin; specific origins only, because auth here is header-based, so a wildcard + credentials would let any site's JS probe the API

Gmail token encryption at rest

backend/app/gmail/crypto.py
Fernet, keyed by GOOGLE_TOKEN_ENCRYPTION_KEY, hard-required in a declared-production environment

Profile-ownership enforcement once authenticated

backend/app/auth.py:156-196
Once a valid session is presented, X-Profile-Id can never reach a different account's profile — mismatch is a hard 403

OAuth CSRF state

backend/app/gmail/oauth.py
secrets.token_urlsafe(32), 10-minute TTL, single-use via delete-on-consume, persisted in a real table

Minimal OAuth scope

backend/app/gmail/oauth.py:64
Exactly gmail.send — no read access, no broader Gmail/Google account access requested

Explicit send confirmation

schemas/outreach.py:79-81, service.py:263-267

confirmed: bool is a required field, re-checked in code, not just enforced by the UI

Atomic send transition

outreach/db_store.py:180-201

Conditional UPDATE ... WHERE status='ready' prevents double-send races

No bulk sending, no automation

Confirmed by repo-wide search (none found)

Every email send is one explicit, synchronous, human-confirmed HTTP request

CV file-type validation via content sniffing

cv/validation.py:96-126

Magic-byte inspection, not just trusting the extension or declared Content-Type

CV path-traversal protection

cv/storage.py:68-77

resolve_storage_path() guards against a crafted filename escaping the storage root

CV file path never exposed to clients

services/cv.py:256-257 (public_cv)

Send-time CV retrieval is resolved server-side from a cv_version_id, never from a client-supplied path

Safe, generic error messages

services/query.py:9-30

Real Postgres error codes/messages/hints are logged server-side only, never returned to the client

Search-term sanitization

services/professors.py:199-206

Strips characters with special meaning in Postgres ilike/or_ filter syntax before building a query from user input

2. Known Security Gaps / Production Hardening — Only What the Code Shows

These are not hypothetical — each one is grounded in a specific code location, and several are explicitly self-documented by the codebase's own comments.

GAP 1: X-PROFILE-ID IS STILL TRUSTED AT FACE VALUE WHEN NO AUTHORIZATION HEADER IS PRESENT AT ALL.

This is the headline gap, and backend/app/auth.py:29-32 says so directly: "a caller presenting no Authorization header at all is still trusted on X-Profile-Id alone." The hardening added in auth.py only protects a caller who *does* present a Supabase session; any client that simply omits the header (a script, a direct API call, a stale frontend build) still gets the pre-hardening behavior.

GAP 2: AUTHORIZATION HAS NO DATABASE-LEVEL BACKSTOP.

The backend connects with the Supabase service-role key, which bypasses Row-Level Security entirely (backend/app/supabase_client.py:14). Every isolation guarantee (profile_id filtering) is enforced purely in application code. A single missed profile_id filter in a new endpoint would have no safety net underneath it.

GAP 3: EMAIL TOKEN-ENCRYPTION PRODUCTION GUARD CHECKS ONLY ENVIRONMENT, NOT RENDER.

A different, adjacent guard in the same feature area (backend/app/routers/email.py:49-51) explicitly checks RENDER too, specifically because ENVIRONMENT was once left unset on a real Render deployment. crypto.py was not updated to match. See Chapter 10 §7.

GAP 4: CV FILE STORAGE DEFAULTS TO LOCAL DISK

which the code's own comment (backend/app/cv/storage.py:11-22) calls unsafe on Render's ephemeral filesystem — and the escape hatch (SUPABASE_CV_BUCKET) is unset in every .env file present in this repo. See Chapter 14 §5.

GAP 5: NO VIRUS/MALWARE SCANNING OF UPLOADED CV FILES.

Confirmed by repository-wide search — zero hits for any scanning library or pattern.

GAP 6: OUTREACH_DRAFTS.STATUS HAS NO DATABASE CHECK CONSTRAINT

The enum is enforced only in application code (backend/migrations/20260909_outreach_drafts_gmail.sql:23-24, comment-only). save_draft() also has no state-machine guard (backend/app/outreach/service.py:117-140) — it will write any status value a caller sends, including jumping a fresh draft directly to "sent" via a raw PATCH. See Chapter 9 §1.

GAP 7: OUTREACH_DRAFTS.PROFESSOR_ID HAS NO FOREIGN-KEY CONSTRAINT

to the professors table (20260909_outreach_drafts_gmail.sql:14) — the database will not stop a draft from referencing a nonexistent professor id; only application code checks this.

GAP 8: NO RATE LIMITING WAS FOUND ANYWHERE IN THE CODEBASE

No middleware, no per-IP or per-profile throttling on any endpoint, including CV upload, matching queries, or email generation/send. NOT VERIFIED FROM CURRENT CODEBASE as an intentional omission or a hosting-platform-level mitigation outside the repo (e.g. Render's own infra); only that no application-level rate limiting exists in this code.

GAP 9: ANTI-FABRICATION IN AI-GENERATED EMAILS IS PREVENTATIVE ONLY, NOT VERIFIED.

The LLM email-generation path restricts what data it's given and instructs it not to fabricate, but there is no post-generation check that the output actually matches the supplied evidence. See Chapter 9 §6.

GAP 10: PROFILES.LINKED_USER_ID — THE COLUMN THE CURRENT AUTH HARDENING DEPENDS ON — WAS, AT LEAST AT ONE POINT DURING DEVELOPMENT, NOT YET APPLIED TO THE LIVE PRODUCTION DATABASE

discovered while diagnosing a real bug (per a test-file comment: "*confirmed by direct introspection of the live database while diagnosing the CV selection 'Database query failed' bug,*" backend/tests/test_auth.py:118-124). The code degrades gracefully when this column is missing (app/auth.py's is_missing_column_error handling), but whether the migration has since been applied to the current live database is NOT VERIFIABLE from this repository alone.

3. What This Adds Up To

AN HONEST SUMMARY

The system's security model is best summarized honestly as: **real, server-verified authentication when a client sends a session token, graceful-but-permissive fallback when it doesn't; authorization enforced entirely in application code with no database backstop; and a small number of specific, self-documented gaps that the codebase's own comments already flag as known trade-offs rather than accidental oversights.** That combination — genuine hardening work visible in the commit history, paired with candid comments about what's still incomplete — is a more accurate and more defensible story to tell a supervisor or interviewer than either "it's fully secure" or "it's insecure": it's a system in the middle of a deliberate, incremental hardening effort, with the exact remaining gap named in its own source code.

16. Failure Scenarios

Source: docs/architecture/failure-scenarios.md — each row traces: trigger → detection → fallback/response → user-visible result → database state.

CV Pipeline

Trigger	Detection	Fallback	User-visible result	DB state
Unsupported file extension or content doesn't match extension	cv/validation.py magic-byte sniff	None — rejected before storage	HTTP 415	No cv_versions row created
File over 10 MB or empty	cv/validation.py:46-52	None	HTTP 413/415	No row created
Format valid but content corrupt (bad PDF body, encrypted PDF, empty text)	try/except in services/cv.py:210-222	File is still saved; extraction marked failed	HTTP 200, parsing_status: "failed", generic message	cv_versions row created with parsing_status="failed"
LLM CV-extraction call fails (network/API error)	Broad except Exception in upload_and_parse	No fallback to heuristic — the whole extraction is marked failed	Same generic failure message; the real cause is only in server logs	cv_versions row created, parsing_status="failed"
Profile confirm attempted on a failed-parse CV	confirm_profile(), services/cv.py:353-354	Blocked outright	HTTP 400 with the stored parse-error message	No normalized profiles/student_* write happens

Matching

Trigger	Detection	Fallback	User-visible result	DB state
Matching requested with no confirmed CV	_confirmed_student(), services/matching.py:111-129	None	HTTP 400	N/A (read-only endpoint)
No professors clear min_score/filters	Scoring + filter pipeline naturally returns an empty list	None needed — normal empty result, not an error	Frontend shows an empty-state ("no matches")	N/A
Opportunity mode requested but no professor has a usable non-closed opportunity	Filtered out in services/matching.py	Professor simply excluded from opportunity-mode results	Fewer or zero results in that mode specifically; unaffected in research mode	N/A

Email Generation

Trigger	Detection	Fallback	User-visible result	DB state
CV not confirmed/failed at generation time	_confirmed_student(), outreach/service.py:506-523	None	HTTP 400	No draft created
Selected opportunity is closed	_load_opportunity_context(), service.py:598-619	None	HTTP 400	No draft created
LLM email generation call fails	try/except inside LlmEmailProvider.generate(), outreach/llm.py:117-142	Silently falls back to the deterministic template provider	User sees a normal-looking generated draft — no error surfaced at all	Draft created and stored with generation_provider="deterministic", indistinguishable from the true no-LLM-configured default
Draft save (store.put()) fails	No explicit try/except around this call in generate_draft()	None identified — would surface as an unhandled exception	Likely a generic 500 (NOT VERIFIED whether a global FastAPI exception handler catches this)	Draft not persisted

Gmail OAuth

Trigger	Detection	Fallback	User-visible result	DB state
User denies consent on Google's screen	Google redirects with ?error=	None	Redirect to /outreach/gmail-callback-error?reason=denied	No row written to gmail_connections
OAuth state missing, already consumed, or expired (>10 min)	consume_state(), gmail/oauth.py:121-168	None — request rejected	Redirect with ?reason=invalid_state	No connection written; the state row is already gone either way
Token exchange with Google fails	try/except around the exchange call, routers/gmail.py	None	Redirect with ?reason=exchange_failed	No connection written
Google returns no access token	Explicit check in the callback	None	Redirect with ?reason=no_token	No connection written
Writing the connection to the DB fails	try/except around the upsert_connection() call	None	Redirect with ?reason=store_failed	Tokens were successfully obtained from Google but never persisted — the user must reconnect

Gmail Send

Trigger	Detection	Fallback	User-visible result	DB state
Gmail access token expiring within 60s	get_valid_access_token(), gmail/service.py:133-201	Automatic refresh using the stored refresh token	None visible — transparent	gmail_connections.token_expires_at, access_token_encrypted updated
Refresh token rejected by Google (400/401 — revoked/invalid)	Same function	Connection marked unusable	HTTP 401, "please reconnect Gmail"	gmail_connections.revoked_at set; draft marked status="failed" via _mark_failed()
Gmail API messages.send returns 400/401/403	GmailApiError wrapper, gmail/client.py	None — send attempt stops	Generic safe HTTP 502 (no raw Google error text leaked)	Draft marked status="failed" with extracted error_code/error_message
CV file missing from storage at send time	read_cv_bytes() raises	Send aborted	Error surfaced to user	Draft marked failed
Two send requests race for the same draft	atomically_set_sending() CAS, outreach/db_store.py:180-201	The losing request's conditional UPDATE affects 0 rows	HTTP 409 Conflict to the loser	Only the winner's transition to sending (then sent/failed) actually happens
confirmed flag omitted or false	Pydantic required-field validation, then an explicit code re-check (service.py:263-267)	Send never starts	HTTP 400/422	No status change — draft stays exactly as it was

General

- **No draft can ever get permanently stuck in "sending."** Every failure branch in the send pipeline routes through _mark_failed(), which transitions the draft to a terminal failed state — the only two terminal outcomes of a send attempt are sent or failed.
- **No automatic retries exist anywhere** — a failed send, a failed generation, or a failed CV parse all require the user to take the next explicit action (re-upload, re-generate, retry send) themselves. There is no background job that revisits a failure.

17. API Map

Source: docs/architecture/api-map.md — every row is grounded in a specific router file.

Universities

Router: backend/app/routers/universities.py

- **GET** /universities — List all universities. Auth: none. Service: services/universities.py::list_universities. Output: UniversityListResponse.
- **GET** /universities/{university_id} — Get one university. Auth: none. Errors: 404 if not found.
- **GET** /universities/{university_id}/departments — List a university's departments. Auth: none. Errors: 404 if university not found.

Departments

Router: backend/app/routers/departments.py — confirmed not called directly by the frontend service layer.

- **GET** /departments/{department_id} — Get one department. Auth: none. Errors: 404.
- **GET** /departments/{department_id}/labs — List a department's labs. Auth: none. Errors: 404 if department not found.

Labs

Router: backend/app/routers/labs.py — same "not called directly by frontend" note as Departments.

- **GET** /labs/{lab_id} — Get one lab. Auth: none. Errors: 404.
- **GET** /labs/{lab_id}/professors — List professors in a lab. Auth: none. Errors: 404 if lab not found.

Professors

Router: backend/app/routers/professors.py

- **GET** /professors/search — Search/filter professors. Input (query params): q, research_area, country, university_id, department_id, recruiting, internship, ra, masters, phd, limit (1-100, default 50). Auth: none. Service: services/professors.py::search_professors. Output: ProfessorSearchResponse.
- **GET** /professors/{professor_id} — Full professor detail (with lab/department/university/research areas joined). Auth: none. Errors: 404.
- **GET** /professors/{professor_id}/research-areas — List a professor's tagged research areas. Auth: none. Errors: 404 if professor not found.

CV

Router: backend/app/routers/cv.py — full pipeline detail: Chapter 7.

- **POST** /cv/upload — Upload a CV file, triggering validation, storage, and extraction. Input: multipart file + optional existing profile_id. Auth: auth.resolve_identity — can create a new profile if none exists. Service: services/cv.py::upload_and_parse. Output: CvUploadResponse. Errors: 413 (too large), 415 (unsupported type).
- **GET** /cv/{cv_id} — Fetch one CV version's metadata + extracted profile. Auth: optional profile_id scoping. Errors: 404.
- **POST** /cv/{cv_id}/parse — Re-run extraction on an already-stored file without re-uploading. Service: services/cv.py::parse_cv. Errors: 400 if the stored file is missing.

Profile

Router: backend/app/routers/profile.py

- **GET** /profile — Get the current profile's active CV + extracted data. Auth: auth.require_profile_id (401 if none, 404 if authenticated with no profile yet). Service: cv_service.get_active_profile.
- **PUT** /profile — Update the extracted-profile fields before confirming. Body: StudentProfileUpdate. Auth: require_profile_id. Service: cv_service.update_extracted_profile.
- **POST** /profile/confirm — Lock in the extracted profile, normalizing it into profiles/student_projects/student_publications/student_research_areas. Auth: require_profile_id. Service: cv_service.confirm_profile. Errors: 400 if no CV or if the active CV's last parse failed.

Matching

Router: backend/app/routers/matching.py

- **GET** /matching/professors — Rank professors by research fit and/or opportunity fit. Input (query): mode (research|opportunity|both), limit (1-100, default 25), university_id, min_score (0-100), email_only, opportunity_type, opportunity_status. Auth: require_profile_id. Errors: 400 if the profile's CV isn't confirmed. Full detail: Chapter 8.

Outreach

Router: backend/app/routers/outreach.py — full detail: Chapter 9.

- **POST** /outreach/drafts/generate — Generate a new email draft for a professor. Body: professor_id, email_type, opportunity_id?. Auth: require_profile_id. Errors: 400 (CV not confirmed / opportunity closed), 404 (professor not found).

- **PATCH** /outreach/drafts/{draft_id} — Edit subject/body and/or set status. Body: SaveDraftRequest{subject, body, status}. Auth: ownership-checked against profile_id.
- **GET** /outreach/drafts/{draft_id}/preview — Preview the rendered send (validates sendability without sending).
- **POST** /outreach/drafts/{draft_id}/attach-cv — Attach a specific cv_version_id to the draft. Errors: validates CV ownership + file existence.
- **GET** /outreach/cv-versions — List the profile's CV versions (for the attach-CV picker).
- **POST** /outreach/drafts/{draft_id}/send — Send the draft via Gmail. Body: SendDraftRequest{confirmed: bool} (required). Errors: 400 (not confirmed / validation failure), 409 (already sending — CAS conflict), 401 (Gmail not connected/token invalid), 502 (Gmail API error). Full detail: Chapter 11.
- **GET** /outreach/history — List past drafts/sends for the profile.

Gmail

Router: backend/app/routers/gmail.py — full OAuth detail: Chapter 10.

- **GET** /gmail/connect — Begin the OAuth handshake. Input: profile_id (header or query), optional return_to. Auth: cannot use a verified session (full-page navigation); accepts X-Profile-Id/query directly. Response: 302 to Google.
- **GET** /gmail/callback — OAuth redirect target. Input: code, state (or error). Response: 302 to a frontend SPA route with success or ?reason=... on failure.
- **GET** /gmail/status — Connection status for the current profile. Auth: require_profile_id. Output: {connected, email} only — never tokens (routers/gmail.py:231-237).
- **POST** /gmail/disconnect — Disconnect Gmail: best-effort token revocation with Google, clears stored tokens, preserves historical sent-draft records (routers/gmail.py:244-255). Auth: require_profile_id. Output: {disconnected: true, message}.

Health/Root

backend/app/main.py

- **GET** / — API identity — links to /docs and /health.
- **GET** /health — Pings the universities table via Supabase. Errors: 503 if the database is unreachable.

18. Interview Q&A

Source: docs/ARCHITECTURE.md, Section 21 — "How I would explain this in an interview"

Q1. Explain FindMyProfessor's architecture.

Short: React SPA talking to a FastAPI backend, which is the only thing that touches Supabase Postgres, Supabase Auth, Google's OAuth/Gmail API, and an optional LLM endpoint. No ORM — direct Supabase query builder calls. No RLS — authorization is entirely application-code enforced.

Deeper: See Chapter 1's 2-minute explanation and Chapter 13 (Backend).

Q2. Why React + FastAPI?

Short: A fast-iterating SPA for a form-heavy, multi-step UI (onboarding → matches → compose), paired with a typed, async-friendly Python backend where the CV-parsing and scoring logic needed real string/regex/date processing — not something you'd want to write in a thinner backend.

Deeper: The CV pipeline (backend/app/cv/) and matching engine (backend/app/matching/) are pure Python with dependency-injected providers (app/cv/extraction/factory.py, app/outreach/factory.py) — straightforward in FastAPI, would need more scaffolding elsewhere.

Q3. How does CV analysis work?

Short: Format-specific text extraction (pypdf/python-docx/olefile/odfpy/striptrf depending on file type), then a structured-field parser — regex/heuristic by default, an optional LLM behind an env var.

Deeper: Chapter 7 (CV Pipeline), files: backend/app/cv/parsers/registry.py, backend/app/cv/extraction/factory.py, heuristic.py, llm.py.

Q4. How do you match students with professors?

Short: A weighted overlap score across four signal types — explicit interests (55%), inferred signals (20%), project/publication evidence (20%), and free-text corroboration (5%) — with weights renormalized across whichever signal types the student actually has data for.

Deeper: Chapter 8 (Matching Engine), file: backend/app/matching/scoring.py. Be ready to walk through the worked example (score = 67) by hand.

Q5. Is matching AI-based?

Short: No — zero ML, zero embeddings, purely deterministic set-intersection scoring. That's a deliberate choice, not a limitation I'm apologizing for.

Deeper: Explain why (see Q6).

Q6. Why deterministic matching?

Short: Reproducibility and explainability — a student can see exactly which research areas overlapped and why a score is what it is, with zero API cost and zero risk of an LLM inventing a match that isn't real.

Deeper: The same score_professor() function is reused verbatim by the email generator (outreach/service.py::_build_match_context), so the matching UI and the generated email can never disagree about what the "real" overlap is.

Q7. How is email personalization done?

Short: A generation-provider abstraction — a deterministic template provider by default, or an LLM provider if an API key is configured — both fed only pre-verified, matching-scored evidence, never the raw CV or the professor's full bio.

Deeper: Chapter 9 (Email Generation), file: backend/app/outreach/llm.py:48-97 for the exact prompt-construction fields.

Q8. How does Gmail OAuth work?

Short: Standard authorization-code flow, gmail.send-only scope, CSRF state persisted in a DB table (not memory, so it survives redeploy/multiple workers) with a 10-minute TTL and single-use enforcement via delete-on-read.

Deeper: Chapter 10 (Gmail OAuth), file: backend/app/gmail/oauth.py.

Q9. Authentication vs. authorization — what's the difference here?

Short: Authentication is 'who is this' — verified via a real round-trip to Supabase's Auth server when a session token is present. Authorization is 'what can they touch' — enforced entirely in application code by filtering on profile_id, with no database-level backstop, because the backend uses the service-role key which bypasses RLS.

Deeper: Chapter 6 (Auth vs Authorization) — and be ready to state the known gap directly: no Authorization header at all still means the legacy X-Profile-Id-trust path is used.

Q10. Why gmail.send instead of full Gmail access?

Short: Least privilege — the app only ever needs to send one email on the student's behalf; it has no legitimate reason to read their inbox, so it doesn't ask to.

Deeper: backend/app/gmail/oauth.py:64 — literally the only scope string in the codebase.

Q11. How are Gmail tokens secured?

Short: Encrypted at rest with Fernet, keyed by an environment variable that's hard-required in a declared production environment.

Deeper: Chapter 10 §7 — and be ready to name the one real gap: that production check uses ENVIRONMENT, not the RENDER variable a sibling check uses, after ENVIRONMENT was once left unset on a real deploy.

Q12. What happens when a Gmail token expires?

Short: It's refreshed lazily, right before a send, with a 60-second buffer. If Google rejects the refresh (revoked/invalid), the connection is marked revoked and the user is told to reconnect.

Deeper: Chapter 10 §9.

Q13. How do you prevent sending without user confirmation?

Short: A required (no-default) confirmed boolean, checked both by schema validation and again explicitly in code, plus an atomic database compare-and-swap on the draft's status so a double-click can't send twice.

Deeper: Chapter 11 §2-3.

Q14. How do you prevent fabricated professor information?

Short: By restricting what data physically reaches the generation step, not by checking the output afterward — the LLM path only ever sees pre-verified, matched fields, never the professor's raw bio or the student's full CV.

Deeper: Chapter 9 §4-6 — and be honest that there is no post-generation fact-check; that's a real, stated limitation.

Q15. How is the CV attached [to a sent email]?

Short: The frontend only ever holds an opaque cv_version_id; the backend resolves that to an actual file path and reads the bytes server-side at send time — the storage path itself never appears in any API response.

Deeper: Chapter 11 §4.

Q16. How is user data isolated?

Short: Every query filters by profile_id, enforced in application code. There's no Row-Level Security backstop because the backend connects with the service-role key.

Deeper: Chapter 5 §RLS.

Q17. What are the current architectural limitations?

Short: No DB-level authorization backstop, local-disk-default CV storage that's unsafe on the inferred hosting platform unless a specific env var is set, no post-generation hallucination check, and a legacy unauthenticated-header path that's only partially closed.

Deeper: Chapter 15 §2 — this is the single best section to have memorized before an interview.

Q18. What would you improve next?

Short: Close the remaining X-Profile-Id-without-a-session gap, align the token-encryption production guard with the RENDER-aware guard used elsewhere, and switch default CV storage to the Supabase Storage bucket path.

Deeper: Those are exactly the top three items in Chapter 15 §2, items 1, 3, and 4.

Q19. How would you scale this architecture?

Short: The stateless FastAPI process already scales horizontally as-is — OAuth state and Gmail tokens were deliberately moved out of in-memory storage into the database specifically to support multiple workers/instances. The heavier lift would be the matching engine's per-request full scan/score of the professor catalog, which would need caching or pre-computation at a larger catalog size.

Deeper: The gmail_oauth_states migration comment (backend/migrations/20260910_gmail_oauth_states.sql:6-9) states this exact multi-worker reasoning directly.

Q20. What happens from clicking "Send" until the professor receives the email?

Short: Walk through the Gmail Send sequence diagram (Chapter 11 §9): validate → atomic CAS to sending → read CV bytes → refresh token if needed → build MIME → call users.messages.send → update DB to sent/failed.

Deeper: See Chapter 11.

19. Learning Roadmap

Source: docs/ARCHITECTURE.md, Section 22

Level	Focus	Files to read	Concepts	Questions to answer before moving on
1	Overall architecture	backend/app/main.py, frontend/src/App.jsx	Containers, routing, CORS	What are the only two live containers? Why is CORS an explicit allow-list here specifically?
2	Frontend/backend communication	frontend/src/services/api.js, backend/app/auth.py	Header-based identity, dependency injection	What two headers does every API call carry, and what does the backend do differently depending on which are present?
3	Database/data model	Chapter 5, backend/migrations/*.sql	RLS vs. app-layer authorization, upsert-based seeding	Why does profiles.id equal an auth.users.id? Why is RLS not used here?
4	CV pipeline	backend/app/cv/validation.py, parsers/registry.py, extraction/factory.py	Content sniffing, provider abstraction	What determines heuristic vs. LLM extraction? What happens to the file vs. the DB row when parsing fails?
5	Matching engine	backend/app/matching/scoring.py, tests/test_matching_scoring.py	Weighted renormalization, coverage scoring	Why does a student with only one signal type still get a full-scale score? What's the one hardcoded generic term?
6	Email generation	backend/app/outreach/service.py, llm.py, deterministic.py	Provider abstraction, evidence reuse	Which function does outreach reuse from matching, and why does that matter? What happens on LLM failure?
7	Authentication	backend/app/auth.py in full	JWT verification, profile claiming	What's the one remaining gap the code itself documents?
8	Gmail OAuth	backend/app/gmail/oauth.py, crypto.py	CSRF state, token encryption	Why was OAuth state moved out of memory into a table? What's the encryption-key production guard's blind spot?
9	Gmail API sending	backend/app/gmail/client.py, outreach/service.py::send_draft	MIME construction, atomic CAS	How is a double-send prevented at the database level?
10	Deployment/security	Chapter 14, Chapter 15	Ephemeral disk risk, missing manifests	What hosting platform is inferred, and from what evidence specifically (not just a comment)?

20. Architecture Glossary

Source: docs/ARCHITECTURE.md, Section 23

Term	In FindMyProfessor's context
Router	A FastAPI APIRouter in backend/app/routers/ — declares HTTP paths/methods, delegates to a service function immediately; contains no business logic itself
Schema	A Pydantic model in backend/app/schemas/ — the typed contract for a request body or response shape
Service	A plain Python module in backend/app/services/ (or outreach/service.py, gmail/service.py) doing the actual orchestration — DB reads/writes, calling into pure-logic modules like matching/scoring.py
Provider (abstraction)	The EmailGenerationProvider/CV-extraction-provider pattern: an abstract interface with two swappable implementations (deterministic and LLM), chosen at runtime by an environment variable
Fallback	Specifically: LLM generation failure silently falling back to the deterministic provider — recorded identically to "LLM never configured," which is a real observability gap here
Deterministic matching	Matching computed by a pure function of set intersections and fixed weights — same inputs always produce the same score, no randomness, no model inference
Renormalization	In matching/scoring.py::combine_components — redistributing a missing signal type's weight proportionally across the signal types that do have data, rather than treating a missing signal as a zero
RLS (Row-Level Security)	A Postgres feature this project does not rely on — the backend's service-role key bypasses it, so all isolation is application-code enforced
JWT	The Supabase Auth session token, verified server-side via supabase.auth.get_user(token) — a real check against Supabase's Auth server, not a locally-decoded/unverified token
OAuth State	A one-time, 10-minute-TTL, database-persisted CSRF token used only during the Gmail connect handshake — unrelated to the Supabase session JWT
Scope (OAuth)	The specific permission being requested from Google — here, exactly gmail.send, nothing broader
Supabase	The hosted Postgres + Auth + Storage platform this project uses for its database, login provider, and (optionally) CV file storage
MIME	The email message format — built here with Python's stdlib email.mime, then base64url-encoded for the Gmail API's messages.send
CV parsing	Two distinct steps conflated in casual speech: (1) text extraction (format → plain text) and (2) structured-field extraction (plain text → named fields)
CORS	Configured here as an explicit origin allow-list, never a wildcard, specifically because this app's identity model is header-based rather than cookie-based
SPA	The React frontend — a single HTML shell where react-router-dom handles all navigation client-side
Idempotent (ingestion)	The /data → database seeder can be re-run safely — it diffs against existing rows and only writes what actually changed
Atomic CAS (compare-and-swap)	The UPDATE outreach_drafts SET status='sending' WHERE status='ready' pattern that prevents a draft from being sent twice

21. Final Cheat Sheets

Source: docs/ARCHITECTURE.md — "Final Cheat Sheets"

A

One-Page Architecture Cheat Sheet

- Two live containers: React SPA + FastAPI backend. No ORM. No server-side sessions/cookies — identity travels in headers.
- Five external dependencies: Supabase Postgres, Supabase Auth, Google OAuth/Gmail, optional LLM endpoint, and either local disk or Supabase Storage for CV files.
- No deployment manifest in-repo; backend hosting on Render is strongly inferred (a functional RENDER env-var check exists), frontend host unconfirmed.
- Authorization = application code filtering by profile_id; there is no database RLS backstop.

B

One-Page Request/Data-Flow Cheat Sheet

HTTP request → CORSMiddleware (main.py) → Router → auth dependency (require_profile_id/resolve_identity) → Pydantic schema validation → Service function → services/query.py::execute() → Supabase (service-role key) → Postgres → Pydantic response schema → JSON.

Every response error is generic/safe; real Postgres errors are logged server-side only.

C

One-Page Auth + Gmail OAuth Cheat Sheet

Two headers on every request: X-Profile-Id + Authorization: Bearer <JWT>. No session → trust the header (legacy path, documented gap). Valid session → header can never reach another account; unlinked legacy profiles get auto-claimed once.

Gmail is a *separate* grant: gmail.send-only scope, encrypted token pair in gmail_connections, CSRF state in gmail_oauth_states (10-min TTL, single-use via delete).

D

One-Page Matching-Engine Cheat Sheet

score = combine_components({interests: 0.55, signals: 0.20, artifacts: 0.20, corroboration: 0.05}), each component = 100 * |student n professor| / |student|, weights **renormalized** across whichever components have data.

One hardcoded generic term stripped: "computer science". Priority: >=80 high, >=50 normal, else low.

Opportunity scoring is separate: open=100/upcoming=80/unknown=50 base, +5 international, +10 undergrad-detected, capped at 100, best-single-opportunity only (no stacking). Zero ML anywhere.

E

One-Page CV Pipeline Cheat Sheet

Upload → 3-layer validation (extension + magic-byte sniff + cross-check) → store (local disk by default, unsafe per the code's own comment unless SUPABASE_CV_BUCKET is set) → format-specific text extraction (pypdf/python-docx/olefile/odfpy/stripf) → structured extraction (**heuristic by default**; LLM only if LLM_API_KEY set, no fallback between the two) → full JSON blob stored on every upload → normalized tables written only on explicit /profile/confirm.

Matching only ever consumes research_interests/research_signals/projects/publications — never skills, education, or identity.

F

One-Page Email-Generation Cheat Sheet

Lifecycle: generated → edited/ready → sending (atomic CAS) → sent | failed. Evidence reuses matching's exact score_professor(). Prompt gets only verified fields (never raw CV, never professor's free-text bio).

Deterministic provider is pure template substitution — fabrication-proof by construction. LLM provider falls back silently to deterministic on any failure, and that fallback is **indistinguishable** in stored data from "LLM never configured." No post-generation fact-check exists.

Send requires a required confirmed:true flag, checked twice, plus the atomic CAS.

G

Interview Questions and Answers

See Chapter 18 above — 20 pre-written short + deep answers with file citations.

H

Things I Should Inspect in the Code Myself

1. Whether backend/migrations/20260910_profiles_linked_user_id.sql has actually been applied to the current live Supabase project (a past test comment found it hadn't been, at least once) — this directly determines whether the auth-hardening in auth.py is fully active in production right now.
2. Whether SUPABASE_CV_BUCKET is set in the actual production environment (not just absent from the .env files present in this repo) — this determines whether CV files actually survive redeploys today.
3. Whether GOOGLE_TOKEN_ENCRYPTION_KEY and ENVIRONMENT are both correctly set in the live Render service — the flagged gap in crypto.py only manifests if both happen to be unset simultaneously.
4. Whether outreach_drafts.selected_opportunity_id is genuinely never populated by generate_draft(), and if so, whether that's a real bug worth fixing.
5. Whether student_projects/student_publications/student_research_areas are read anywhere this pass didn't find — if truly unread, that's persisted-but-dead data worth either wiring up or removing.
6. Whether any rate limiting exists at the hosting-platform level (outside this repo) even though none exists in application code.